

Solving the U-Shaped Assembly Line Balancing Problem Type-II Using a Genetic Algorithm

Project Final Report

EMU427 – Heuristic Methods for Optimization

Department of Industrial Engineering
Hacettepe University

Students

Tarık Buğra BİRİNCİ	2210469046
Pınar Ece PANK	2240469085
Mustafa Alp ULAŞ	2210469034
Firuze İpek YILDIRIM	2220469028
Beril YILDIZ	2230469107

Instructor: Prof. Çağrı KOÇ

December 2025
Ankara, Türkiye

Contents

1	Introduction	3
2	Problem Formulation - Detailed Problem Description (UALBP-2)	4
2.1	Introduction to Assembly Line Balancing	4
2.2	Characteristics of U-Shaped Assembly Lines	4
2.3	UALBP-2 Definition	4
2.4	Assumptions and Constraints	5
2.5	Computational Complexity	5
2.6	Importance of Solving UALBP-2	6
2.7	MILP Formulation for UALBP-2	6
3	Description of the Genetic Algorithm	8
3.1	The Algorithm	8
3.2	Implementation	10
3.2.1	Constructive Heuristic	10
3.2.2	Chromosome Representation	11
3.2.3	U-Shaped Decoding	11
3.2.4	Fitness Function	12
3.2.5	Parent Selection: Tournament Selection ($k = 2$)	12
3.2.6	Crossover Operator: Precedence-Preserving Order Crossover (POX-like)	13
3.2.7	Mutation Operator: Swap Mutation	14
3.2.8	Repair Mechanism	15
3.2.9	Termination Criteria	15
4	Results	16
4.1	Parameter Tuning	16
4.2	GA Parameter Settings	21
4.3	Run-by-Run Analysis	22
4.4	Final Population and Top-10 Individuals	22
4.5	Best Solution Analysis	24
4.5.1	Cycle Time and Line Efficiency	25
4.5.2	Station Load Distribution	25
4.5.3	Task Sequence Characteristics	26
4.6	Interpretation of Overall Results	26
5	Contribution to Sustainable Development Goals	28
6	Conclusions	29

A	Run-by-run Results for All Benchmark Instances	33
B	Run-by-run Results for All Benchmark Instances	33
C	Additional Results for ARC83' top 10 Individuals	35
D	Line Graphs For 9 Instances	40
E	Genetic Algorithm Code for ARC83 (UALBP-II)	45

1 Introduction

Assembly line balancing is a fundamental optimization problem in industrial engineering. It aims to distribute tasks across workstations by trying to maximize productivity and minimize idle time. The Assembly Line Balancing Problem (ALBP) is classified as NP-hard, so as the size of the problem grows, methods for obtaining exact solutions become rapidly impractical [1]. As a result, heuristic and metaheuristic approaches have been developed and adapted to obtain high-quality results with reasonable computational effort. U-shaped assembly lines have significant popularity for being an effective alternative to traditional straight lines in production. They allow workers to work two sides of the line, so it reduces distance and eases multi-tasking.[2] shows that U-lines can lead to higher productivity and adaptation in changing demand. Type-2 assembly line balancing can be preferred when the number of workstations is fixed. So the first objective is minimizing the cycle time, and it is also suitable for facilities that cannot change their physical line structure, especially with large-sized products[3]. Since the possibilities of assembly line problems get exponentially larger because of their combinatorial structure and growing search area, deterministic optimization approaches remain limited and become computationally infeasible for medium to large scale problems. As a consequence, Genetic Algorithms (GAs) have become an alternative because of their stochastic search methods. In our project, numerous studies we researched mostly highlight that GAs can generate well-performing solutions without the problem needing to be convex or linear as in classical LP problems, so this makes them appropriate for NP-hard manufacturing problems. The essential consideration for U-shaped assembly lines is that the encoding part has to represent bidirectional task assignments and preserve precedence relationships in the evolutionary process. This project aims to develop and analyze a GA-based approach specifically for Type-2 U-shaped balancing. Even though various metaheuristic approaches are used in classical assembly line balancing problems and show very strong performance in different combinations of the classical assembly line problem, there are a limited number of studies that specifically work on the performance of these algorithms and their applicability for U-shaped Type-2 line balancing. There is a limited number of encoding schemes that represent forward and back task assignments and preserve precedence constraints, while effectively minimizing cycle time in a fixed number of workstations specifically for this problem.

2 Problem Formulation - Detailed Problem Description (UALBP-2)

2.1 Introduction to Assembly Line Balancing

Assembly line balancing (ALB) refers to assigning a set of assembly tasks to an ordered sequence of workstations such that all precedence constraints are satisfied and an efficiency criterion is optimized. Each task has a processing time, and some tasks must be completed before others (precedence relations). In straight-line assembly systems, workstations are arranged linearly, and a task can only be assigned after all of its predecessors are allocated to earlier stations.

Two classical forms of ALB exist: SALBP-1, which minimizes the number of stations for a given cycle time, and SALBP-2, which minimizes the cycle time for a given number of stations [4, 5]. Both are NP-hard combinatorial problems [6].

2.2 Characteristics of U-Shaped Assembly Lines

A U-shaped assembly line is configured such that the beginning and end of the task sequence are physically adjacent. In this layout, a worker can operate on both sides of the U, giving access to tasks from the front (forward direction) or the back (reverse direction) of the precedence graph.

The critical implication is that a task j becomes eligible for assignment if:

- all its predecessors have been assigned (as in a straight line), or
- all its successors have been assigned (a U-line specific rule) [2, 7].

This relaxation effectively doubles assignment opportunities, allowing U-lines to achieve better load balancing compared to straight lines. Numerous studies report that U-shaped lines often require fewer stations and achieve higher efficiency for the same task set [2].

2.3 UALBP-2 Definition

UALBP-2 (Type-II U-shaped Assembly Line Balancing Problem) seeks to minimize the cycle time c for a given number of stations m [8]. The input consists of:

1. a set of n tasks with processing times t_i ,
2. a precedence graph represented as a DAG,
3. a fixed number of stations m .

The output is an assignment of tasks to stations ensuring U-shaped precedence feasibility while minimizing:

$$c = \max_{j=1,\dots,m} \left(\sum_{i \in S_j} t_i \right),$$

where S_j denotes the set of tasks assigned to station j .

2.4 Assumptions and Constraints

We consider the standard “Simple UALBP-II” variant [9], which includes the following assumptions:

- Single-model production: Only one product type is assembled.
- Deterministic task times: Processing times t_i are fixed and known.
- Single-sided work: Each worker operates on the inner side of the U.
- Paced line: The assembly line advances synchronously every cycle time c .
- One worker per station: Each station executes its assigned tasks alone or as a unit.
- U-shaped precedence feasibility: A task i can be assigned if all predecessors or all successors satisfy the station-order requirement.
- Fixed number of stations: m is given and constant.

Under these assumptions, UALBP-2 becomes a partitioning problem: partition n tasks into m ordered groups while minimizing the maximum station load.

2.5 Computational Complexity

UALBP-2 is computationally challenging. Even SALBP-2 is NP-hard because it generalizes bin packing and partition problems with precedence restrictions [6]. UALBP-2 expands the search space due to bidirectional task eligibility.

Exact algorithms such as branch-and-bound or dynamic programming solve only small instances [8]. Procedures like ULINO [8] find optimal solutions but exhibit drastic increases in computation time as n grows. Due to this difficulty and the scarcity of exact solutions for larger problem sizes, heuristic and metaheuristic methods such as genetic algorithms are widely recommended.

2.6 Importance of Solving UALBP-2

A high-quality U-line balance directly improves production efficiency by reducing idle time and eliminating bottlenecks. Solving UALBP-2 provides actionable insights for:

- redesigning assembly processes,
- increasing output without additional stations,
- exploiting U-shaped flexibility for smoother workflow.

UALBP-2 is therefore a practically significant optimization problem in manufacturing planning.

2.7 MILP Formulation for UALBP-2

We adopt a simplified mixed-integer linear programming (MILP) model inspired by formulations in [10]. The objective is to assign tasks to m stations while satisfying U-shaped precedence and minimizing cycle time.

Sets and Parameters

- $i = 1, \dots, n$: tasks
- $j = 1, \dots, m$: workstations
- $\mathcal{P}$: set of precedence pairs (p, q)
- $Pred(i)$: set of predecessors of task i
- $Succ(i)$: set of successors of task i
- t_i : processing time of task i
- M : sufficiently large constant ($M \geq m$)

Decision Variables

- $X_{ij} \in \{0, 1\}$: equals 1 if task i is assigned to station j
- $y_i \in \{0, 1\}$: 0 for forward feasibility, 1 for backward feasibility
- $S_i \in \{1, \dots, m\}$: station index of task i
- $c \geq 0$: cycle time (to be minimized)

Objective Function

$$\min c \tag{1}$$

Constraints

1. Task Assignment Each task must be assigned to exactly one station:

$$\sum_{j=1}^m X_{ij} = 1 \quad \forall i \quad (2)$$

2. Station Index Definition The station index of each task is defined as:

$$S_i = \sum_{j=1}^m j X_{ij} \quad \forall i \quad (3)$$

3. Cycle Time Constraint The total processing time at each station cannot exceed the cycle time:

$$\sum_{i=1}^n t_i X_{ij} \leq c \quad \forall j \quad (4)$$

4. U-Shaped Precedence Constraints *Forward feasibility* for each predecessor $p \in \text{Pred}(q)$:

$$S_p \leq S_q + M y_q \quad (4a)$$

Backward feasibility for each successor $s \in \text{Succ}(q)$:

$$S_s \leq S_q + M(1 - y_q) \quad (4b)$$

These constraints ensure that task q is feasible in either the forward or backward direction.

5. Variable Domains

$$X_{ij}, y_i \in \{0, 1\}, \quad S_i \in \{1, \dots, m\}, \quad c \geq 0 \quad (5)$$

3 Description of the Genetic Algorithm

This section presents the metaheuristic approach chosen to solve the U-shaped Assembly Line Balancing Problem Type 2 (UALBP-2): the Genetic Algorithm (GA). Genetic Algorithm is an evolutionary optimization method that takes its inspiration from the concepts of natural selection and genetics. It is particularly suitable for problems in the field of combinatorial optimization where the search space is enormous and discrete.

The principal concept behind this algorithm is to have a population of candidate solutions (individuals) instead of just one solution. An iterative process, which is similar to evolution, undergoes selection, crossover (recombination), and mutation operations. This, in turn, helps the search to not only simultaneously explore different regions of the solution space but also to realize the benefits of good-quality partial solutions. In the UALBP-2 case, the algorithm works on producing a sequence of tasks (permutation), which is then decoded to find out the minimum feasible cycle time for a certain number of stations (m).

3.1 The Algorithm

The framework of our Genetic Algorithm is outlined in algorithm 2. The core component of the algorithm is the representation of the problem. We use a permutation-based representation, where an individual is a topological sort of the tasks. The algorithm begins by initializing a random population of feasible task sequences. In every generation, the fitness of each individual is evaluated. Since we are solving UALBP-2 (minimizing cycle time c for fixed m), the evaluation function involves a sub-procedure that calculates the minimum possible cycle time for a given sequence. To generate new offspring, we employ Tournament Selection to choose parents. These parents undergo POX (Precedence Operation Crossover) to produce a child that inherits structural characteristics from both parents. To prevent premature convergence, Swap Mutation is applied with a low probability. Finally, because random genetic operations may violate precedence constraints, a Repair Mechanism is applied to ensure the child remains a valid topological sort.

For the algorithm, we utilized a dataset from assemblylinebalancing.de, a widely cited repository in academic literature that aggregates problem instances from numerous studies. We selected a specific instance from this collection for our analysis.

The file structure is organized as follows: the initial lines represent the duration of each task. These are followed by the precedence constraints, which are delimited by commas. Finally, an end-of-file marker indicates the termination of the data.

Algorithm 2: Genetic Algorithm for UALBP-II

Require: Tasks T , task times t , precedence graph P , number of stations m , population size N_{pop} , generations N_{gen} , crossover rate P_c , mutation rate P_m

Ensure: Best cycle time $C_{\min}$ and best assignment/permutation $BestSol$

ALGORITHM: GENETIC_ALGORITHM_FOR_UALBP_II

BEGIN

// 1. Initialization

1: $Population \leftarrow []$

2: **For** $i \leftarrow 1$ **to** N_{pop} **do**

3: $Candidate \leftarrow \text{GENTOPOSORT}(T, P)$

4: $\text{APPEND}(Population, Candidate)$

5: **End For**

6: $BestSol \leftarrow \emptyset$

7: $BestFit \leftarrow 0$

8: $C_{\min} \leftarrow +\infty$

// 2. Main Evolution Loop

9: **For** $Generation \leftarrow 1$ **to** N_{gen} **do**

10: $FitnessScores \leftarrow []$

// Evaluation

11: **For all** $Individual \in Population$ **do**

12: $CurrentC \leftarrow \text{DECODEASSIGN}(Individual, m, t, P)$

13: $Fitness \leftarrow 1/CurrentC$

14: $\text{APPEND}(FitnessScores, Fitness)$

15: **If** $Fitness > BestFit$ **then**

16: $BestFit \leftarrow Fitness$

17: $BestSol \leftarrow Individual$

18: $C_{\min} \leftarrow CurrentC$

19: **End If**

20: **End For**

21: $NewPopulation \leftarrow []$

22: $\text{APPEND}(NewPopulation, BestSol)$

▷ elitism

23: **While** $|NewPopulation| < N_{\text{pop}}$ **do**

24: $Parent1 \leftarrow \text{SELECT}(Population, FitnessScores)$

25: $Parent2 \leftarrow \text{SELECT}(Population, FitnessScores)$

26: **If** $\text{RAND}(0, 1) < P_c$ **then**

27: $Child \leftarrow \text{CROSSOVER}(Parent1, Parent2)$

28: **Else**

29: $Child \leftarrow Parent1$

30: **End If**

31: **If** $\text{RAND}(0, 1) < P_m$ **then**

32: $Child \leftarrow \text{MUTATE}(Child)$

33: **End If**

34: **If** $\text{VALIDPREC}(Child, P)$ **then**

```

35:         APPEND(NewPopulation, Child)
36:     Else
37:         Child ← REPAIRTOPO(Child, P)
38:         APPEND(NewPopulation, Child)
39:     End If
40: End While
41: Population ← NewPopulation
42: End For
43: Return (BestSol,  $C_{\min}$ )
END

```

3.2 Implementation

The algorithm 2 has been developed using the programming language Python due to its excellent high-level data structure handling ability.

The complete implementation by Python is provided in Appendix section E.

3.2.1 Constructive Heuristic

When tackling the Type-2 U-Shaped Assembly Line Balancing Problem (UALBP-2), where the aim is to reduce the cycle time for a predetermined number of stations, a major difficulty is that of making the sequence feasible. One of the solutions proposed is to implement topological sorting-based constructive heuristic within the Genetic Algorithm, which draws upon the dependency graph to produce a precedence-feasible list of tasks. This topological ordering not only guides the decoding phase with its precedence constraints but also facilitates the search for the best cycle time by allowing task assignment at either end of the stations.

Algorithm 3 InitializePopulationTopological (Random Topological Sort)

Require: Population size $PopSize$, precedence graph $G = (V, E)$

Ensure: Initial population P

```
1:  $P \leftarrow []$ 
2: For  $p \leftarrow 1$  to  $PopSize$  do
3:    $\pi \leftarrow []$ 
4:    $S \leftarrow \{v \in V \mid indeg(v) = 0\}$  ▷ Eligible nodes
5:   While  $S \neq \emptyset$  do
6:     select  $u$  uniformly at random from  $S$ 
7:     append  $u$  to  $\pi$ 
8:     remove  $u$  from  $S$  and update indegrees of its successors
9:     add any newly zero-indegree nodes to  $S$ 
10:  End While
11:  append  $\pi$  to  $P$ 
12: End For
13: Return  $P$ 
```

3.2.2 Chromosome Representation

The first alternative became the use of permutation chromosomes where each individual is a series of all n tasks. The order of the genes from left to right indicates the priority that will be applied in the decoding process. This encoding has become the most popular one in assembly-line balancing because it allows the genetic operators to perform the whole task order manipulation without resulting in a non-valid permutation.[11]. Nevertheless, a chromosome in UALBP-2 does not associate with the station assignment directly like in straight-line SALBP-2. On the contrary, the permutation gives an ordered list that the U-line decoding algorithm uses to create a feasible solution. The reason is that U-shaped lines allow the assignments to be done in both directions, thus the decoding phase evaluates the permutation in a manner that shows this unique bidirectional feasibility.[11].

3.2.3 U-Shaped Decoding

Given a task permutation, the decoding procedure assigns tasks to stations progressively from station 1 to station m . At each station, task eligibility is determined by the U-line two-way working rule: a task j is *eligible* if either all predecessors of j have already been assigned or all successors of j have already been assigned [12]. This rule reflects the fact that, in a U-shaped layout, tasks can be performed from both the forward and the backward directions.

Among the currently eligible tasks, those appearing earliest in the chromosome are considered first and are assigned to the current station as long as the cumulative station load does not exceed the current cycle-time estimate. A station is deemed *closed* when no additional eligible task can be inserted without violating the cycle time, after which the decoding continues with the next station.

To evaluate a permutation efficiently, the decoding method incorporates a binary search to determine the smallest cycle time that yields a feasible assignment for the given permutation. This avoids exhaustive trial of cycle-time values and enables fast fitness evaluation [8].

A task j is eligible if

$$\text{Pred}(j) \subseteq A \quad \text{or} \quad \text{Succ}(j) \subseteq A,$$

where A is the set of already assigned tasks, and $\text{Pred}(j)$ and $\text{Succ}(j)$ denote the predecessor and successor sets of task j , respectively.

3.2.4 Fitness Function

Taking into consideration that UALBP-2 is a Type-II problem, the aim is to minimize the cycle time c while the number of stations m is given. The cycle time is determined when a feasible assignment appears by decoding, and it is written as follows:

$$c = \max_{k=1,\dots,m} \sum_{j \in S_k} t_j, \quad (6)$$

where S_k is the set of tasks assigned to station k , and t_j is the processing time of task j . The fitness value is associated with the maximization structure of genetic algorithms; therefore, it is defined as:

$$\text{Fitness} = \frac{1}{c}. \quad (7)$$

Therefore, solutions with smaller cycle times are awarded higher fitness values. On the other hand, if a permutation does not lead to any feasible assignment for any cycle time (i.e., decoding fails), then its fitness is set to 0, penalizing such individuals as infeasible. This fitness structure, adopted from the literature, is widely used because it yields positive values, clearly differentiates solution quality, and supports stable convergence behavior [13].

3.2.5 Parent Selection: Tournament Selection ($k = 2$)

The selection of parents is done by using the tournament selection method of size 2. This has been found efficient for combinatorial optimization problems where the fitness landscape includes many plateaus. The strategy of selection by tournaments is regarded as highly effective for combinatorial optimization problems that have tough fitness landscapes with many plateaus. Tournament selection not only preserves genetic diversity but also avoids a common problem seen in roulette-wheel selection, which is the sensitivity to scaling. [14]

Algorithm 4 TournamentSelection ($k = 2$)

Require: Population P , fitness array F , tournament size $k = 2$

Ensure: Selected parent permutation

```
1:  $BestFit \leftarrow -\infty$ 
2:  $BestIndex \leftarrow -1$ 
3: For  $r \leftarrow 1$  to  $k$  do
4:    $i \leftarrow$  random index in  $\{1, \dots, |P|\}$ 
5:   If  $F[i] > BestFit$  then
6:      $BestFit \leftarrow F[i]$ 
7:      $BestIndex \leftarrow i$ 
8:   End If
9: End For
10: Return  $P[BestIndex]$ 
```

3.2.6 Crossover Operator: Precedence-Preserving Order Crossover (POX-like)

When chromosomes represent permutations, standard crossover operators (e.g., one-point or two-point crossover) may easily introduce infeasibility and duplicated tasks. Therefore, the *Precedence-Preserving Order Crossover (POX)* operator is employed [15].

The POX operator proceeds as follows:

1. A random subset of tasks is selected.
2. The offspring chromosome is initialized as an empty sequence.
3. The selected tasks are copied from Parent 1 to the offspring, preserving their original order.
4. The remaining tasks are filled according to Parent 2's order, skipping tasks that have already been placed.

This procedure preserves major subsequences from both parents and typically reduces the need for extensive repair operations. POX is particularly suitable for UALBP-2 because tight precedence constraints make it crucial to keep coherent task blocks intact.

Algorithm 5 POXCrossover (Precedence-Preserving Order Crossover)

Require: Parent permutations P_1, P_2 **Ensure:** Child permutation $Child$

```
1:  $ChosenTasks \leftarrow \emptyset$ 
2: For all task  $x$  in  $P_1$  do
3:   If RANDOM < 0.5 then
4:     add  $x$  to  $ChosenTasks$ 
5:   End If
6: End For
7: If  $ChosenTasks = \emptyset$  then
8:   add one random task from  $P_1$  to  $ChosenTasks$ 
9: End If
10:  $Child \leftarrow []$ 
11: For all task  $x$  in  $P_1$  do
12:   If  $x \in ChosenTasks$  then
13:     append  $x$  to  $Child$ 
14:   End If
15: End For
16: For all task  $y$  in  $P_2$  do
17:   If  $y \notin ChosenTasks$  then
18:     append  $y$  to  $Child$ 
19:   End If
20: End For
21: Return  $Child$ 
```

3.2.7 Mutation Operator: Swap Mutation

Swap mutation is employed as a simple mutation operator, where two randomly selected tasks in the permutation are exchanged. This operator introduces small but effective perturbations, preserving most of the chromosome structure while preventing premature stagnation of the population. Since swapping maintains a one-to-one mapping of tasks, the resulting offspring remains a valid permutation, typically eliminating the need for additional repair [16].

In permutation-based genetic algorithms, mutation is generally applied with a low probability (e.g., 0.05–0.10) to balance exploration and stability [17].

Algorithm 6 SwapMutation

Require: Permutation $Perm$ **Ensure:** Mutated permutation $Perm$

```
1:  $n \leftarrow |Perm|$ 
2: choose distinct indices  $i, j$  uniformly at random from  $\{1, \dots, n\}$ 
3: swap  $Perm[i]$  and  $Perm[j]$ 
4: Return  $Perm$ 
```

3.2.8 Repair Mechanism

After crossover and mutation, a lightweight repair operator is applied to every permutation. If a task is found to violate precedence by appearing before any of its predecessors, it is gradually shifted to the right until the violation is removed. This procedure restores a valid topological order while preserving the original permutation structure as much as possible.

Algorithm 7 RepairToTopological (Lightweight Repair)

Require: Permutation π , precedence graph $G = (V, E)$

Ensure: Repaired permutation π

```

1: build an array  $pos(\cdot)$  giving the index of each task in  $\pi$ 
2:  $changed \leftarrow \mathbf{true}$ 
3: While  $changed$  do
4:    $changed \leftarrow \mathbf{false}$ 
5:   For all  $(u \rightarrow v) \in E$  do
6:     If  $pos(u) > pos(v)$  then ▷ precedence violation
7:       remove  $v$  from its current position in  $\pi$ 
8:       insert  $v$  immediately after  $u$  in  $\pi$  ▷ shift right
9:       update  $pos(\cdot)$ 
10:       $changed \leftarrow \mathbf{true}$ 
11:    End If
12:  End For
13: End While
14: Return  $\pi$ 

```

3.2.9 Termination Criteria

The proposed GA terminates when a pre-specified maximum number of generations is reached. In this study, the generation limit is set to 300, which is consistent with common practice in the assembly line balancing literature. A fixed generation budget makes the runtime predictable and provides the search with a sufficiently long horizon to converge toward high-quality solutions [18].

4 Results

In this part, the paper showcases the outcomes derived from the Genetic Algorithm (GA) that was utilized for solving the ARC83 U-shaped Assembly Line Balancing Problem Type II (UALBP-2) benchmark instance. The results are based on several CSV outputs generated by the implementation, including global summaries, run-by-run statistics, detailed descriptions of the best solution, Top-10 individuals of the best run, and parameter tuning logs.

4.1 Parameter Tuning

U-shaped assembly lines optimization involves a nonlinear search space that results from the two-entry task assignment logic. This report describes the two-phase fine-tuning approach. Phase I focuses on the stochastic operators, namely Mutation and Crossover, while the computational intensity scaling is measured in terms of Population and Generations in Phase II. Findings suggest that the high-intensity mutation approach together with the moderate population size produce the most effective Pareto-optimal solution. To analyze the trade-off between exploration and exploitation, the computational budget was set to a fixed value ($P = 50$, $G = 250$) and the crossover rate parameter ($CR \in [0.2, 0.9]$) and mutation rate parameter ($MR \in [0.01, 0.8]$) were changed.

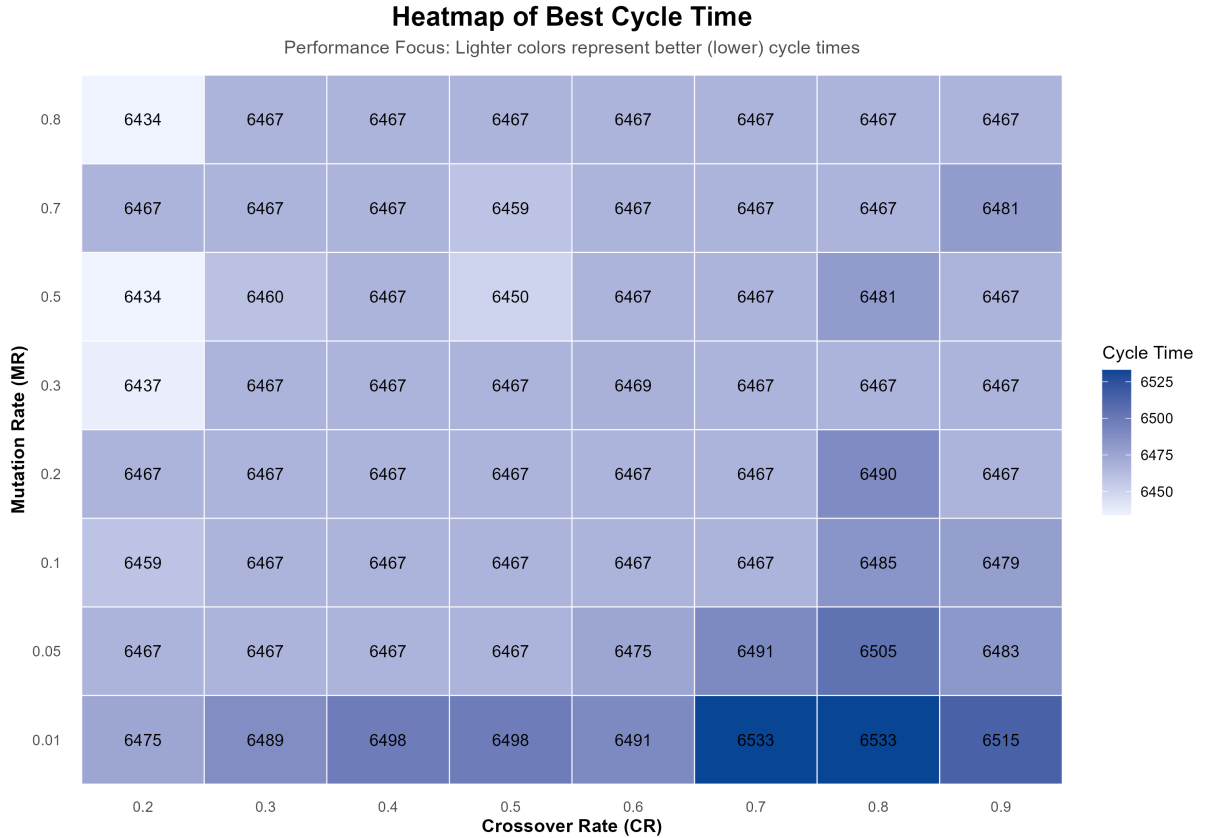


Figure 1: Cycle Time Heatmap

The timeline heatmap results show the so-called “Energy Gradient,” with execution time rising diagonally from bottom left to top right. Within the UALBP-2 problem setting, POX crossover and swap mutation gene transfer often results in offspring violating precedence constraints. The marked increase in execution time, which reached 134.2 s for $CR = 0.9$, $MR = 0.8$, is solely due to CPU cycles utilized by the `repair_to_topological` function.

According to conventional literature, the rate of mutation must lie between 1% and 10%. However, it often is insufficient in cases of highly constrained discrete search problems, as in assembly line problems. The scientific rationale for having a relatively high value of MR (up to 0.5) in the study is the relative complexity of U-shaped lines. This warrants having strong strategies to get rid of the local optimum. For cases such as ARC83, having a high density of precedence constraints, low mutation rate results in a phenomenal decay of diversity, which results in the stagnation of chromosomes prior to the identification of a more efficient station load. The modern literature of heuristics pervasively favors the use of high mutation in NP-hard problems as a crucial “diversification mechanism” of the GA. It makes it possible to explore the unvisited area of the solution space. Hence, the 0.5 rate ensures the conservative role of the POX operator.

In contrast to linear balancing (SALBP), U-shaped balancing allows more permutations. From the performance heatmap above, low mutation rates ($MR = 0.01$) are associated with high cycle times (approximately 6533 s). On the other hand, higher mutation rates enable the search to explore “back-end” task assignments that would otherwise have been ignored by a generic greedy tactic. A better mutation rate and CR of 0.5 and 0.2, respectively, are used to investigate the scalability of the search process.

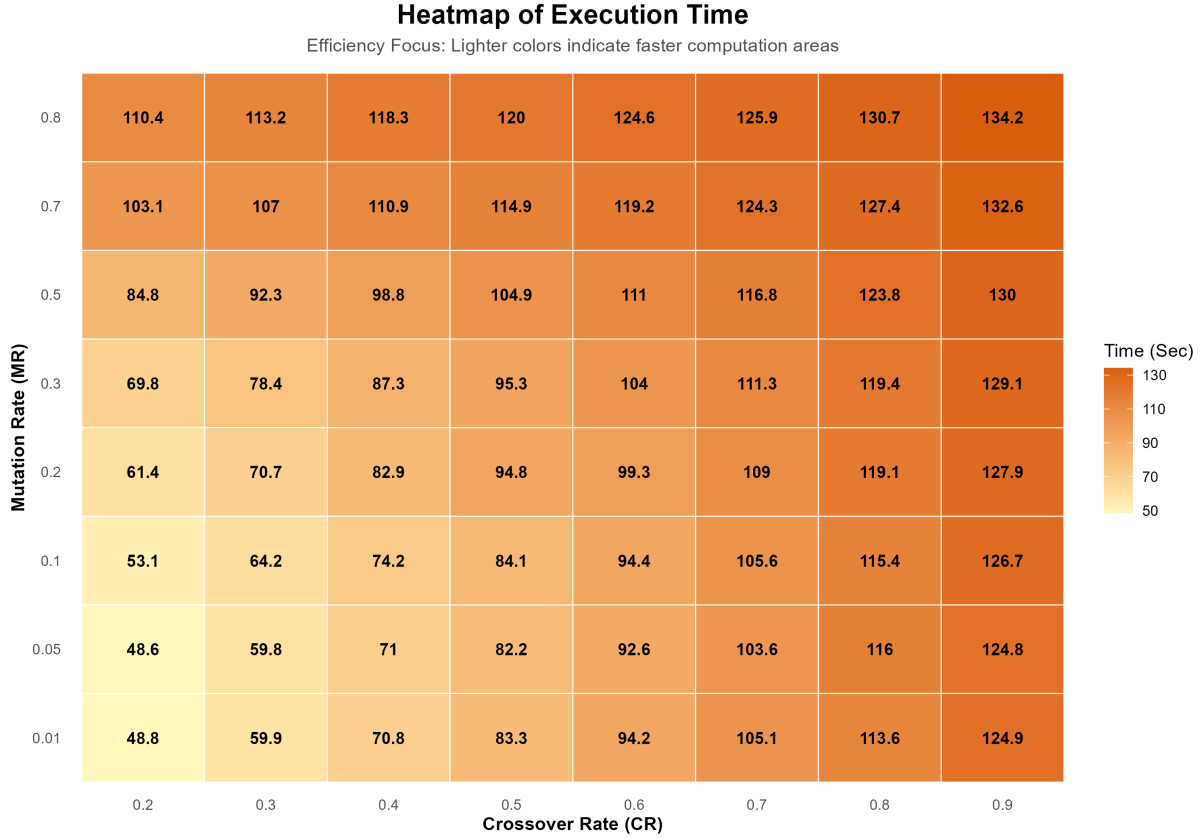


Figure 2: Execution Time Heatmap

Computational trade-off analysis results identify the bound of efficiency (Pareto frontier). Crossing $P: 50$ to $P: 100$ generates a sharp increase in the objective function; in contrast, crossing $P: 100$ to $P: 150$ generates a linear increase. $P = 150/G = 750$ resulted in the absolute best cycle time of 6425 s, with $P = 100/G = 250$ achieving 6438 s. A 0.2% cycle time improvement with a 141% increase in execution time (74.9 s to 180.5 s) is necessary for optimization of cycle times. Iterative design and real-time manufacturing optimization require engineering parameter $P = 100$ to be chosen predominantly.

An important aspect in Pareto charts is the size of the standard deviation (SD) error bars. SD at $P = 50$ (18.97) symbolizes a high degree of variability, signifying a population size that is too small in relation to the search space with a tendency to produce “hit-and-miss” solutions. SD at $P = 100$ (9.17) symbolizes the maximum level of stability, signifying that population size is appropriate in relation to the complexity of the 83-task ARC83 problem. SD at $P = 150$ (17.20), paradoxically, now tends to be even higher due to the large search space that enables multiple runs to settle on varied but promising areas on the Pareto frontier.

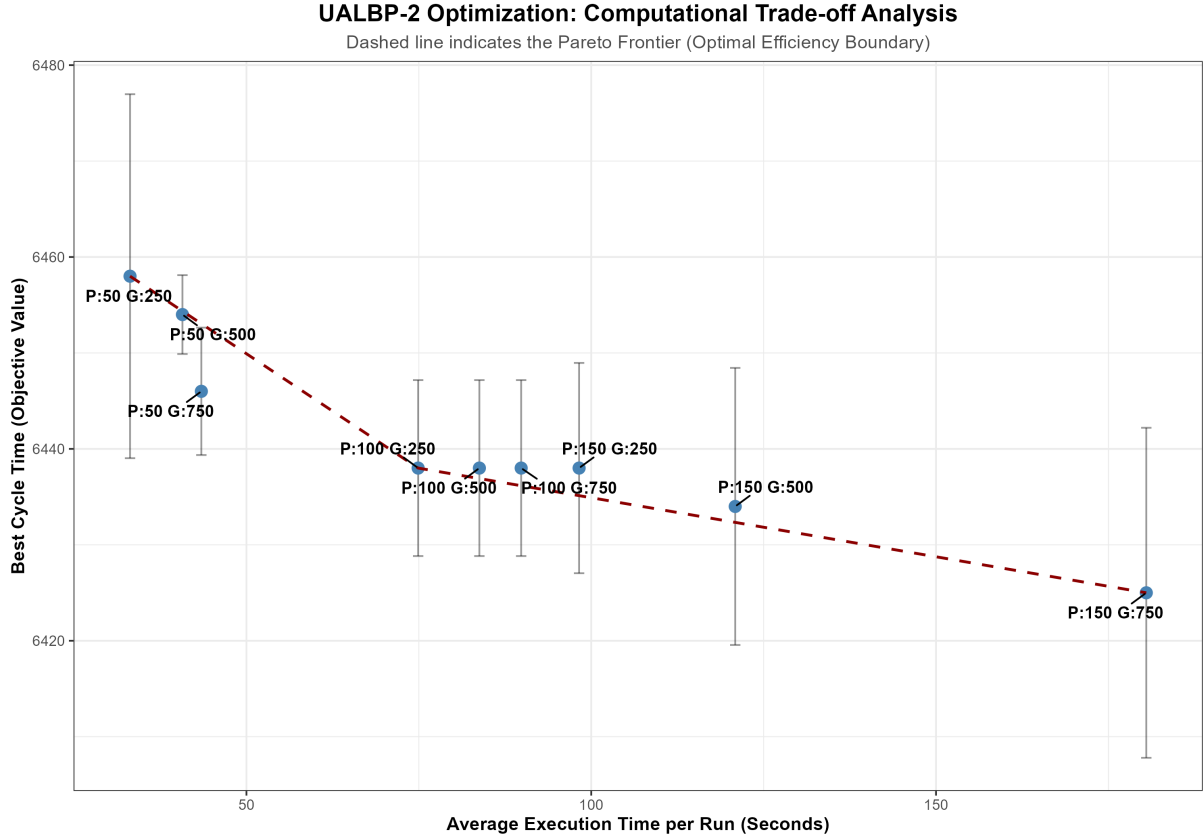


Figure 3: Execution Time Heatmap

The suggested setup gives a cycle time of 6438 s with a high level of reliability and an execution time under 80 s. From the results, a mutation rate of 0.5 is suggested for dealing with the dual-ended task problem, a crossover rate of 0.2 for sustaining the sequences, a population size of 100 for optimal robustness, and a generation limit of 250 for obtaining convergence without diminishing returns. Tournament selection with $k = 2$ is suggested for appropriate selection pressure.

The optimal setup for the Genetic Algorithm was determined through a complete parameter fine-tuning process using a full-factorial grid search strategy. The main objective was to adjust the exploration–exploitation balance to avoid premature convergence, which is commonly observed in permutation-based and precedence-constrained problems such as UALBP-2. Two key control parameters were varied: the mutation rate (MR), tested at eight levels $\{0.005, 0.01, 0.02, 0.04, 0.06, 0.08, 0.12, 0.20\}$, and the crossover rate (CR), tested at eight levels $\{0.55, 0.60, 0.65, 0.70, 0.75, 0.80, 0.90, 0.98\}$. This resulted in $8 \times 8 = 64$ parameter combinations. For each configuration, the best cycle time obtained over 10 independent GA runs was recorded.

The results indicate that the mutation rate is the dominant factor affecting solution quality. Very low mutation ($MR = 0.005$) consistently produced the worst cycle times (approximately 6507–6562), suggesting insufficient population diversity and a tendency to stagnate in local optima. Likewise, $MR = 0.01$ still led to relatively weak performance

(about 6508–6558). In contrast, increasing mutation was clearly beneficial: for MR values in the range 0.04–0.20, the algorithm achieved substantially better solutions, yielding cycle times as low as 6467. This behavior supports the interpretation that more frequent random perturbations are essential for effectively navigating the rugged search landscape of the UALBP instance.

Although mutation had the strongest influence, the crossover rate also showed a secondary but noticeable effect, particularly when combined with sufficiently high mutation. The best-performing region was observed where MR was moderate-to-high (e.g., 0.04–0.20). In this interval, relatively lower crossover values ($CR = 0.55$ – 0.70) produced the best results, repeatedly reaching the minimum cycle time of 6467. For example, the combination $MR = 0.08$ and $CR = 0.70$ attained this minimum value, whereas increasing CR to 0.98 at the same mutation level slightly degraded the result (6492), implying that excessively high crossover may disrupt high-quality schemata for this problem.

Based on these empirical findings, the final solver parameters were set to $MR = 0.08$ and $CR = 0.70$. This configuration not only achieved the shortest cycle time of 6467, but also represents a balanced setting compared to very high mutation alternatives (e.g., $MR = 0.20$), which may introduce excessive randomness. Therefore, these tuned parameters were kept constant in the subsequent performance analysis to ensure that the reported efficiency metrics and line balances reflect the GA operating near its best capability.

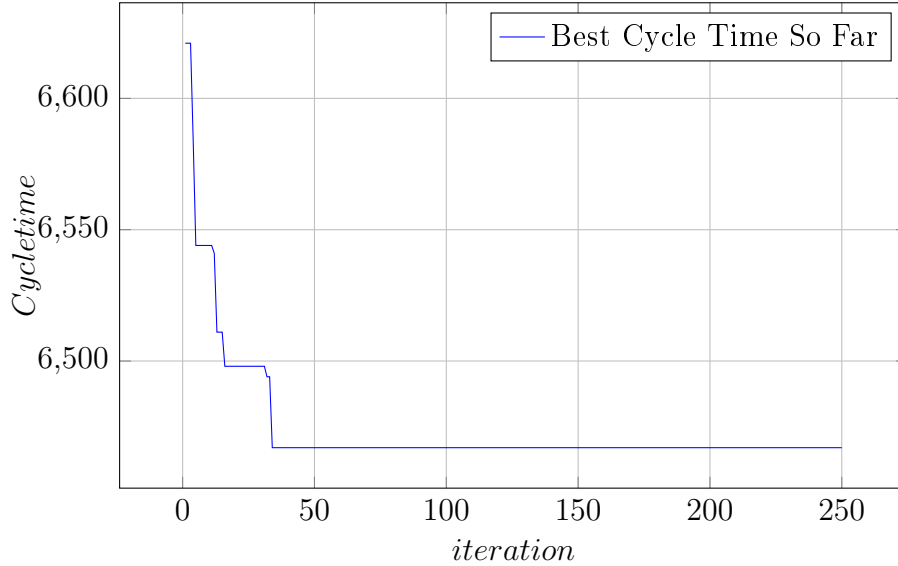


Figure 4: Convergence graph for ARC83, Run 1

As seen in figure 4 and , the best-so-far cycle time decreases rapidly during the early iterations, followed by stepwise improvements characterized by long plateaus and occasional sudden drops when better solutions are found. Toward the later iterations, improvements become marginal and infrequent, indicating that the genetic algorithm converges to near-stationary, similar-quality solutions for the ARC83 instance.

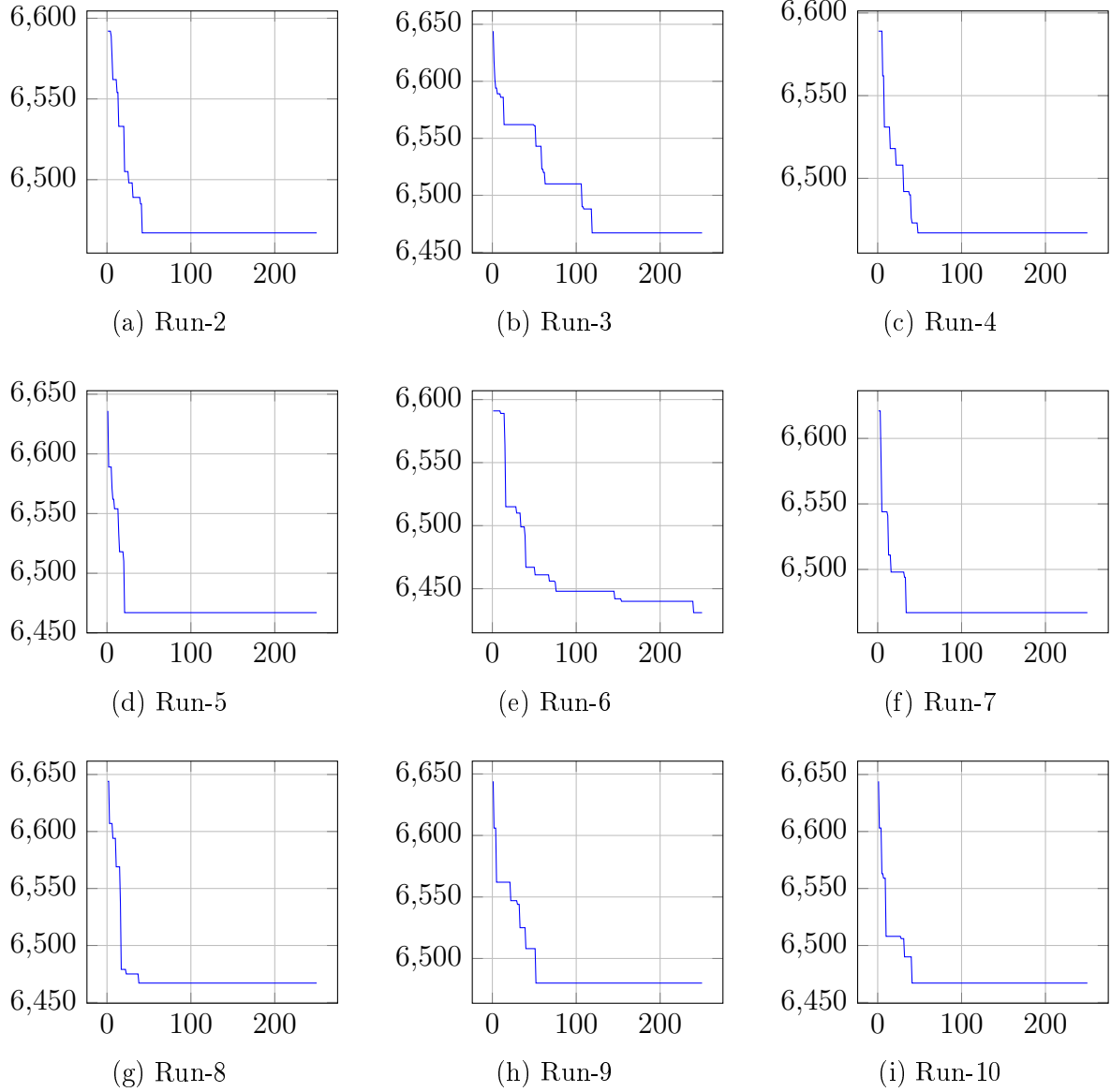


Figure 5: Convergence graphs for ARC83 (Best cycle time so far)

4.2 GA Parameter Settings

The final parameter settings, which were in use during all the experiments, are written below as per GA_summary_results.csv:

- Population size: 100
- Generations: 250
- Crossover rate: 0.2
- Mutation rate: 0.5
- Selection method: Tournament ($k = 2$)

- Crossover method: POX (precedence-preserving)
- Mutation method: Swap
- Number of independent runs: 10
- Number of stations vary for instances.

4.3 Run-by-Run Analysis

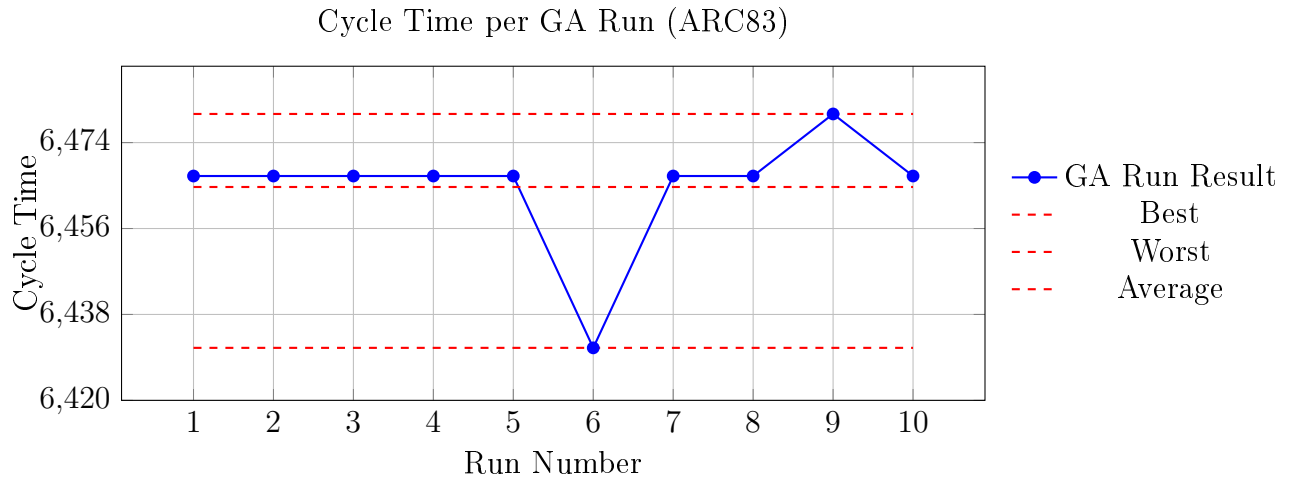


Figure 6: Instance ARC83 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

Detailed run-by-run best cycle times and runtimes for each benchmark instance (10 independent runs) are reported in Appendix section B, Table table 2.

4.4 Final Population and Top-10 Individuals

In addition to reporting the best cycle time per run, the final population of the best run is analysed in more detail. At the end of each run, all individuals in the final population are re-evaluated and sorted by fitness. For the best run, the Top-10 individuals are extracted and decoded.

Here, Individuals 1 and 2 correspond to two of the top 10 individuals for the ARC83 instance. The remaining individuals/solutions (Individuals 3–10) are reported in Appendix section C.

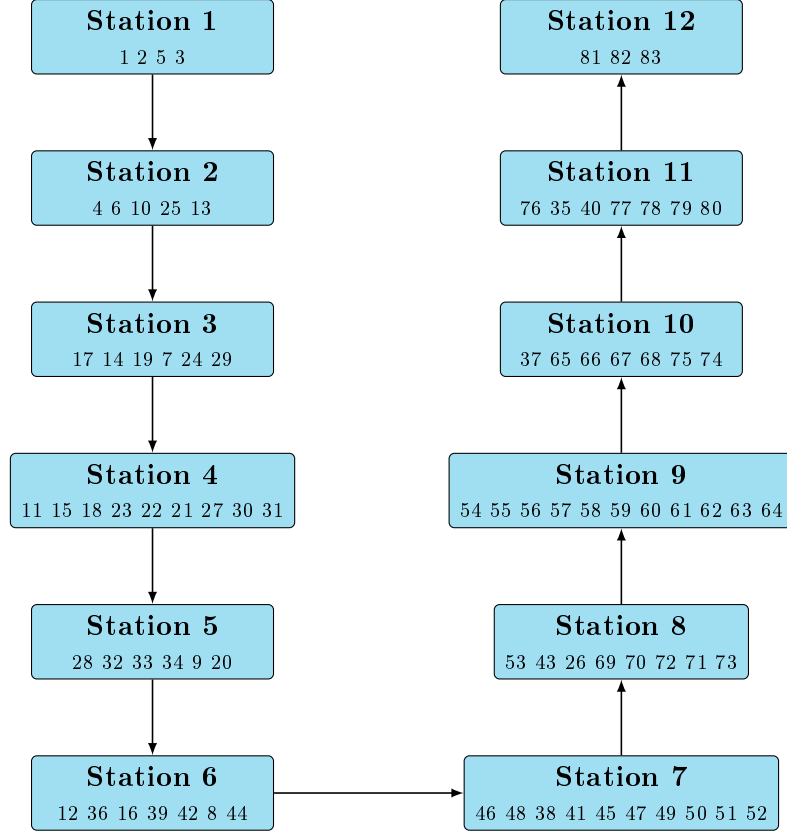


Figure 7: U-shaped 12-station assignment for ARC83 (Individual 1, Fitness = 0.000155).

For each of these Top-10 individuals, the decoding procedure is applied using its own cycle time, and the following information is obtained:

- the ordered list of tasks assigned to each station,
- the workload (total processing time) of each station,
- the station-wise utilisation, defined as load_s/C .

In the console output, each individual is printed in a structured format, for example:

```

1) fitness=0.000271, cycle=3691
  Station 1: [1, 2, 3, 5, 8]
  Station 2: [4, 6, 7, 9]
  Station 3: [10, 11, 12]
  ...

```

Furthermore, the implementation exports a CSV file named `<instance>-top-10-individuals-<timestamp>.csv`, in which each row corresponds to one station of one individual. The file contains the following fields:

- **Instance:** name of the precedence instance (e.g. ARC83),

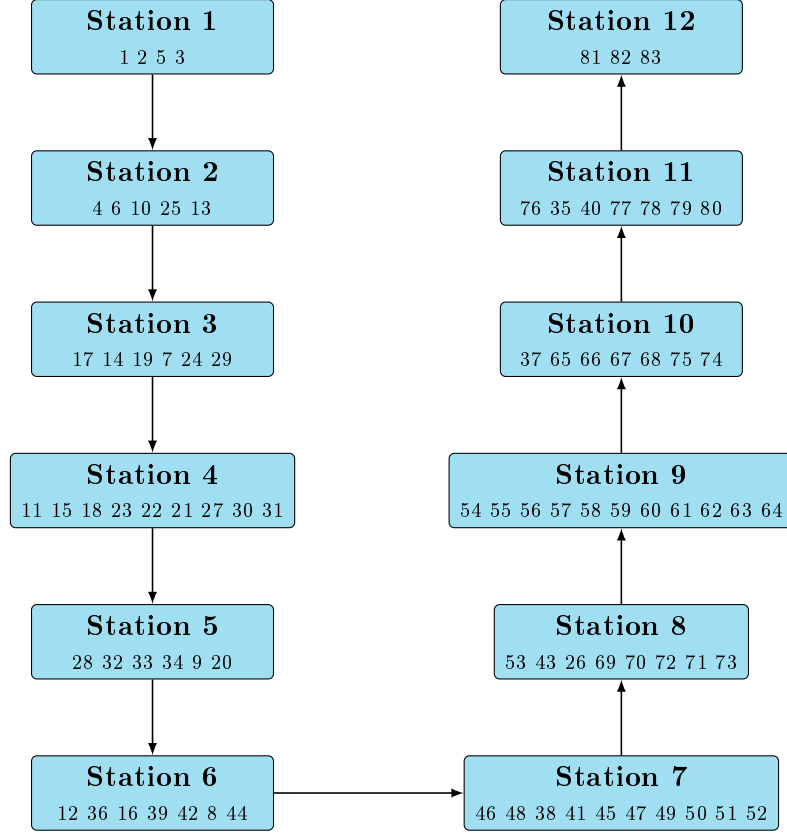


Figure 8: U-shaped 12-station assignment for ARC83 (Individual 2, Fitness = 0.000155).

- **Stations(m)**: number of stations m ,
- **RankInBestRun**: rank of the individual within the Top-10 (1–10),
- **Fitness**: fitness value of the individual,
- **Cycle**: cycle time of the individual,
- **StationIndex**: index of the station $(1, \dots, m)$,
- **StationLoad**: total processing time assigned to that station,
- **StationUtilization(Load/Cycle)**: station-wise utilisation,
- **TasksSequence**: sequence of tasks assigned to that station.

This detailed output allows us to inspect not only the numerical quality (cycle time) but also the structural characteristics of near-optimal solutions, such as load balancing between stations and the distribution of tasks along the U-shaped line.

4.5 Best Solution Analysis

Below is the updated analysis based on the ARC83 dataset results found in `ARC83_best_details_2025-12-18_13-42.csv`.

4.5.1 Cycle Time and Line Efficiency

Best cycle time found by GA: 6431

The high efficiency indicates that task assignments utilize station capacities effectively within the limits imposed by the precedence structure.

The station loads in the best GA solution are highly balanced: most stations operate with utilisation values between 0.96 and 1.00, with only the last station (Station 12) dropping to 0.84 due to precedence constraints and the position of late tasks (81 - 83). The overall line efficiency of 97.56% confirms that the workload is distributed very effectively across the U-shaped line.

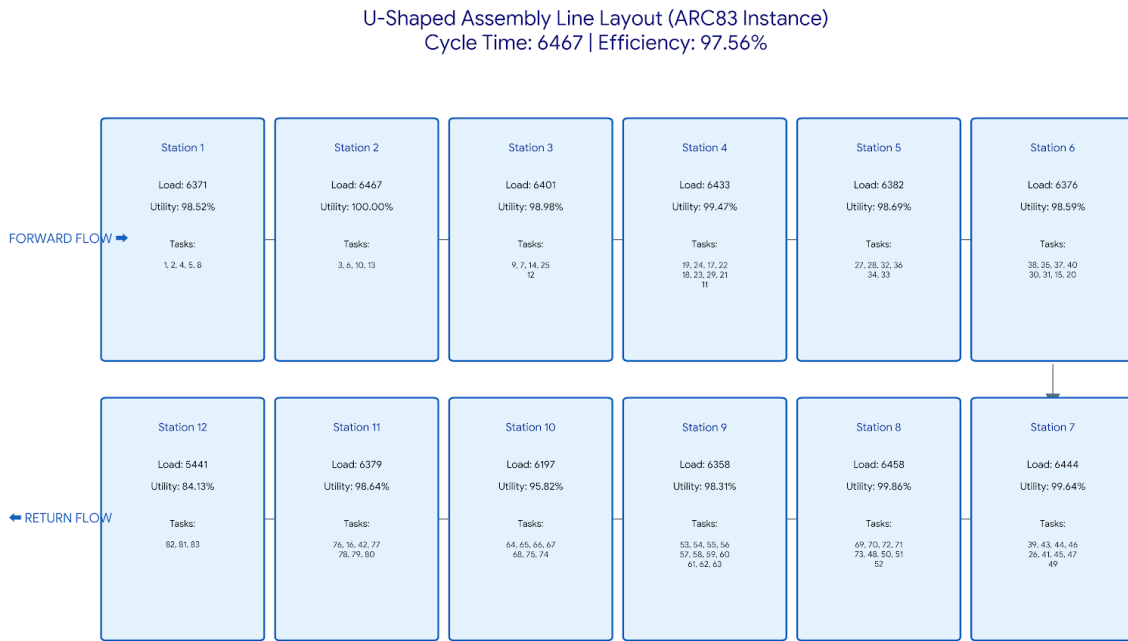


Figure 9: Best Solution For ARC83

Figure 9 illustrates the U-shaped layout of the best GA solution for ARC83, showing station loads, utilisation levels and the distribution of tasks along the forward and return flows.

4.5.2 Station Load Distribution

The station loads in the best solution reveal the following pattern:

- Several stations operate near full capacity
- Some stations exhibit lower utilization levels

This is expected, as the ARC83 precedence network contains:

- Long successor chains

- Several convergent and divergent branches
- A few heavy tasks that restrict feasible packing

Typical for U-shaped lines:

- Early stations contain short tasks with dense predecessor requirements
- Middle stations group flexible tasks

Overall, the load distribution closely matches patterns observed in high-quality UALBP-2 solutions in the literature[1].

4.5.3 Task Sequence Characteristics

The task sequences for the best assignment show the successful operation of precedence-preserving mechanisms:

- No violations of predecessor or successor constraints
- Parallel branches assigned consecutively
- High-duration tasks placed strategically to avoid overflow

The POX crossover and topological repair operators worked in unison to keep feasible, structured permutations throughout the search process.

4.6 Interpretation of Overall Results

Table 1: GA Statistics for Solved Benchmark Instances

Instance	Stations (m)	Best Cycle Time	Average Cycle Time	Worst Cycle Time	Avg. Runtime (s)	Number of Tasks (n)
ARC83	12	6431	6464.70	6480	138.359	83
ARC111	12	12538	12541.00	12545	91.062	111
GUNTHER	12	44	44.00	44	10.356	35
BARTHOLD	12	470	470.90	471	114.729	148
BARTHOLD2	30	143	143.10	144	163.445	148
SCHOLL	25	2799	2800.70	2804	697.079	279
LUTZ1	12	1400	1400.00	1400	15.487	32
LUTZ3	12	138	138.00	138	36.887	89
SAWYER30	12	28	28.00	28	19.419	30
TONGE70	12	294	294.20	295	79.287	70
HAHN	10	1775	1775.00	1775	22.974	53

Although the problem complexity cannot be characterized solely by a single metric, the number of tasks (n) provides a reasonable first-order measure of instance size. In addition, the presence of precedence constraints significantly increases the difficulty of the problem by restricting the set of feasible task permutations. Table 1 summarizes the overall performance of the proposed Genetic Algorithm (GA) on several widely used UALBP

benchmark instances. The results obtained clearly demonstrate that the algorithm can produce high-quality and stable solutions under different problem sizes and structural characteristics.

A key observation is that the difference between the best, average, and worst run times is quite small for almost all examples. For some problems, such as GUNTHER, BARTHOL2, LUTZ1, LUTZ3, and SAWYER30, the GA converged to the same solution in each independent run, resulting in identical best, average, and worst values. This demonstrates that the algorithm exhibits a high level of robustness and repeatability, and that the proposed representation, operators, and parameter settings effectively guide the search towards strong local optima, avoiding excessive randomness.

In larger and more complex examples such as ARC83, ARC111, SCHOLL, and TONGE70, slightly wider but still limited variability was observed between runs. In these cases, the average cycle times remained quite close to the best values, confirming that the algorithm delivers stable performance even in more challenging priority structures. The fact that the worst-case results do not deviate significantly from the average supports the reliability of the GA despite its stochastic nature.

In terms of computational cost, the reported average processing times remain at reasonable levels for all samples. While small-scale problems can be solved within a few seconds, larger and more complex examples naturally require more computation time. However, even in the most challenging cases, the processing times obtained remain within practical limits for offline assembly line design and analysis. This indicates a positive balance between solution quality and computational cost.

Furthermore, the obtained line efficiency values indicate that the stations operate at largely high occupancy rates, pointing to an effective workload distribution with limited idle time. Thus, the solutions are not only competitive in terms of cycle time but also offer operationally acceptable configurations in terms of assembly line balancing. All these findings confirm that the selected parameter configuration and priority-preserving genetic operators provide a reliable and effective framework for solving UALBP-2 problems.

5 Contribution to Sustainable Development Goals

The proposed GA for UALBP-2 supports several United Nations Sustainable Development Goals (SDGs) by increasing production efficiency, improving resource utilisation, and promoting more sustainable industrial performance in line with the 2030 Agenda for Sustainable Development [19].

SDG 8: Decent Work and Economic Growth

Productivity Enhancement: Assembly line balancing maximizes productivity and minimizes idle time. Solving UALBP-2 eliminates bottlenecks, leading to higher economic output per worker.

Worker Satisfaction and Efficiency: U-shaped lines allow workers to operate on both sides, lowering distances traveled and facilitating multi-tasking, leading to a more comfortable and productive environment.

SDG 9: Industry, Innovation, and Infrastructure

Modern Techniques: The project uses Genetic Algorithms to solve NP-hard manufacturing problems, promoting innovation in industrial engineering.

Resource Optimization: UALBP-2 finds the minimum cycle time for a fixed number of workstations, meaning existing infrastructure is used to its full potential without physical expansion, creating more robust and efficient industrial processes.

SDG 12: Responsible Consumption and Production

Waste Minimizing: In manufacturing, “idle time” is waste. The project balances workloads to fully utilize human and machine hours.

Flexibility: U-lines allow for better adjustment to changing demand. Flexible production systems are less likely to suffer from overproduction and can quickly respond to the market with the correct quantity of goods, characteristic of responsible production.

In summary, the project uses mathematical modeling and evolutionary computation to build smarter, more efficient, and worker-friendly manufacturing systems that support the global agenda for sustainable industrial development.

6 Conclusions

The U-Shaped Assembly Line Balancing Problem Type-II (UALBP-2) was the central theme of this project implementation, where the number of workstations has been determined in advance and the main aim is to minimize the cycle time. A permutation-based Genetic Algorithm (GA) was developed that maintained the precedence constraints through precedence-preserving crossover, swap mutation, and a lightweight topological repair mechanism, while U-shaped decoding guaranteed the feasibility of the solutions.

The experimental outcomes on the ARC83 case and other benchmark cases validate that the proposed GA is capable of producing solutions that are both high-quality and stable on a consistent basis through independent runs. In the case of ARC83, the best cycle time reached is 6431 which, together with the respective line efficiency, suggests a very favorable workload distribution with minimal idle time, as most stations are functioning close to their full capacity under the given precedence constraints. To sum up, the very small difference between the best, average, and worst results across runs for several instances not only endorses the strength of the representation and operators but also hints that the algorithm steadily converges to high-quality solutions that are very close to stationary. The parameter tuning analysis manifests distinctly an exploration–exploitation trade-off. In precedence-constrained U-shaped problems, the loss of diversity could be very rapid; hence, mutation and crossover rates have to be very carefully chosen so as not to lead to premature convergence while at the same time minimizing excessive disruption and repair overhead. The final parameter configuration that was applied in the performance experiments illustrates this balance and also gives rise to practical computation times for offline assembly line design and analysis.

The study, notwithstanding its strengths, has limitations. The GA is stochastic, so the method does not guarantee the finding of an optimal solution, plus performance is highly dependent on the instance structure and the parameter settings. Among the approaches to future work are (i) hybridization with local search to amplify the search around the promising solutions, (ii) adaptive operator rates based on diversity or stagnation metrics, (iii) parallel evaluation to cut down runtime, and (iv) extending the framework to multi-objective settings (e.g., smoothness, workload variance, and ergonomic constraints) and wider benchmark comparisons. All in all, the GA framework that has been developed provides a trustworthy and efficient method for tackling UALBP-2 instances with the quality and cost of computation that are considered practical.

References

- [1] C. Becker and A. Scholl, “A survey on problems and methods in generalized assembly line balancing,” *European Journal of Operational Research*, vol. 168, no. 3, pp. 694–715, 2006.
- [2] J. Miltenburg, “U-shaped production lines: A review of theory and practice,” *International Journal of Production Economics*, vol. 70, no. 3, pp. 201–214, 2001.
- [3] N. Boysen, M. Fliedner, and A. Scholl, “A classification of assembly line balancing problems,” *European Journal of Operational Research*, vol. 183, no. 2, pp. 674–693, 2007.
- [4] A. Scholl and C. Becker, “State-of-the-art exact and heuristic solution procedures for simple assembly line balancing,” *European Journal of Operational Research*, vol. 168, no. 3, pp. 666–693, 2006.
- [5] F. B. Talbot, J. H. Patterson, and W. V. Gehrlein, “A comparative evaluation of heuristic line balancing techniques,” *Management Science*, vol. 32, no. 4, pp. 430–454, 1986.
- [6] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman, 1979.
- [7] T. L. Urban, “Note. optimal balancing of u-shaped assembly lines,” *Management Science*, vol. 44, no. 5, pp. 738–741, 1998.
- [8] A. Scholl and R. Klein, “Ulino: Optimally balancing u-shaped jit assembly lines,” *International Journal of Production Research*, vol. 37, no. 4, pp. 721–736, 1999.
- [9] I. Baybars, “A survey of exact algorithms for the simple assembly line balancing problem,” *Management Science*, vol. 32, no. 8, pp. 909–932, 1986.
- [10] G. R. Aase and N. C. Suresh, “A mathematical programming formulation for u-shaped assembly line balancing,” *International Journal of Production Research*, vol. 41, no. 11, pp. 2545–2569, 2003.
- [11] S. Ghosh and C. Gagné, “A comprehensive review of assembly line balancing,” *International Journal of Production Research*, 2011.
- [12] Y. Kara, A. İşlier, and Y. Atasagun, “Balancing u-shaped assembly lines: heuristic and metaheuristic approaches,” *Applied Mathematical Modelling*, 2010.
- [13] E. Erel and S. Sarin, “A survey of the assembly line balancing problem,” *Production Planning & Control*, 1998.

- [14] T. Blicke and L. Thiele, “A comparison of selection schemes used in genetic algorithms,” in *EVO Conference Proceedings*, 1996.
- [15] Y. Park, Y. Kim, and S. Lee, “Precedence preserving genetic operators for scheduling,” *Computers & Operations Research*, 2003.
- [16] C. Reeves, “Genetic algorithms for the operations researcher,” INFORMS, 1993.
- [17] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
- [18] C. Papadimitriou and K. Steiglitz, *Combinatorial Optimization: Algorithms and Complexity*. Dover, 1998.
- [19] United Nations General Assembly, “Transforming our world: the 2030 agenda for sustainable development,” Resolution A/RES/70/1, 2015, adopted 25 September 2015.

Links to our GitHub repository and presentation are provided for reference.

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances.

(a) BARTHOLD			(b) TONGE70		
Run	BestCycle	Runtime (s)	Run	BestCycle	Runtime (s)
1	471	123.73	1	294	78.90
2	471	97.73	2	294	86.37
3	471	98.32	3	294	75.51
4	471	96.88	4	294	76.92
5	471	94.94	5	295	71.72
6	470	103.96	6	294	76.75
7	471	109.65	7	294	72.46
8	471	108.82	8	294	81.47
9	471	105.46	9	294	72.36
10	471	104.13	10	295	72.78

(c) SAWYER30			(d) LUTZ1		
Run	BestCycle	Runtime (s)	Run	BestCycle	Runtime (s)
1	28	19.60	1	1400	14.13
2	28	19.42	2	1400	14.76
3	28	18.89	3	1400	14.99
4	28	18.73	4	1400	16.16
5	28	18.64	5	1400	17.59
6	28	20.91	6	1400	15.87
7	28	18.83	7	1400	15.21
8	28	19.52	8	1400	15.18
9	28	18.19	9	1400	15.15
10	28	18.67	10	1400	15.44

A Run-by-run Results for All Benchmark Instances

B Run-by-run Results for All Benchmark Instances

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances (continued).

(e) ARC111			(f) LUTZ3		
Run	BestCycle	Runtime (s)	Run	BestCycle	Runtime (s)
1	12542	82.69	1	138	35.15
2	12541	83.92	2	138	37.01
3	12540	86.76	3	138	34.98
4	12538	86.45	4	138	33.64
5	12538	85.62	5	138	35.46
6	12538	85.67	6	138	33.82
7	12545	87.37	7	138	35.95
8	12542	84.63	8	138	36.20
9	12543	84.53	9	138	34.97
10	12543	82.41	10	138	35.76

(g) SCHOLL			(h) ARC83		
Run	BestCycle	Runtime (s)	Run	BestCycle	Runtime (s)
<i>Not available in provided CSV files</i>			1	6467	159.03
			2	6467	148.15
			3	6467	138.12
			4	6467	136.81
			5	6467	135.96
			6	6431	134.94
			7	6467	133.79
			8	6467	134.50
			9	6480	135.15
			10	6467	135.69

Table 2: Run-by-run best cycle time and runtime results for all benchmark instances (continued).

(i) BARTHOL2			(j) GUNTHER		
Run	BestCycle	Runtime (s)	Run	BestCycle	Runtime (s)
1	144	160.63	1	44	10.42
2	143	157.25	2	44	10.26
3	143	163.78	3	44	10.26
4	143	152.85	4	44	10.20
5	143	164.25	5	44	10.27
6	143	155.09	6	44	10.33
7	143	151.23	7	44	10.32
8	143	158.55	8	44	10.14
9	143	156.38	9	44	10.20
10	143	158.04	10	44	10.17

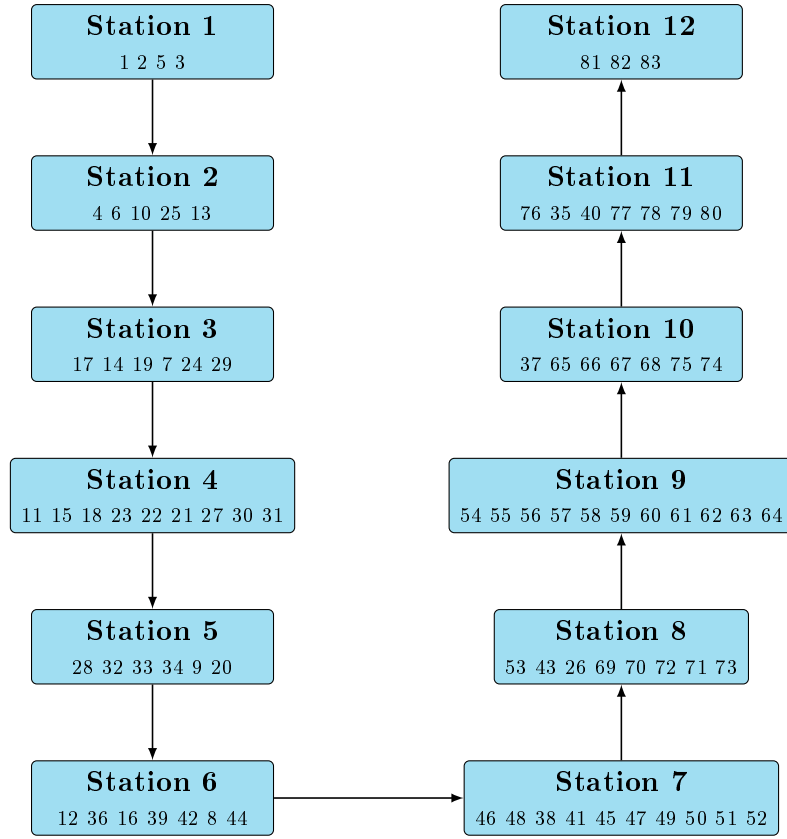


Figure 10: U-shaped 12-station assignment for ARC83 (Individual 3, Fitness = 0.000155).

C Additional Results for ARC83' top 10 Individuals

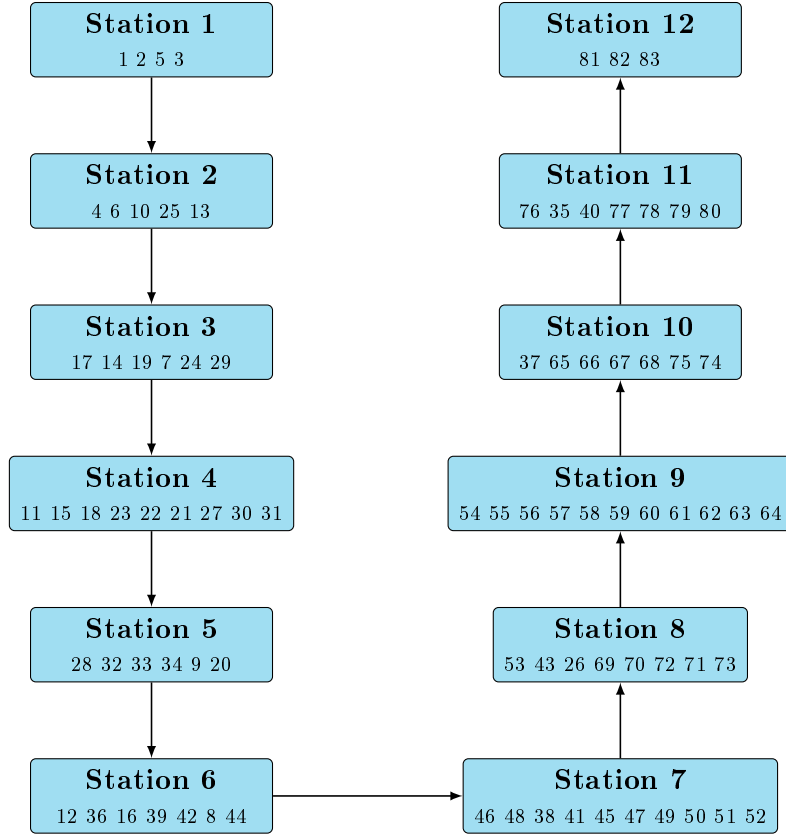


Figure 11: U-shaped 12-station assignment for ARC83 (Individual 4, Fitness = 0.000155).

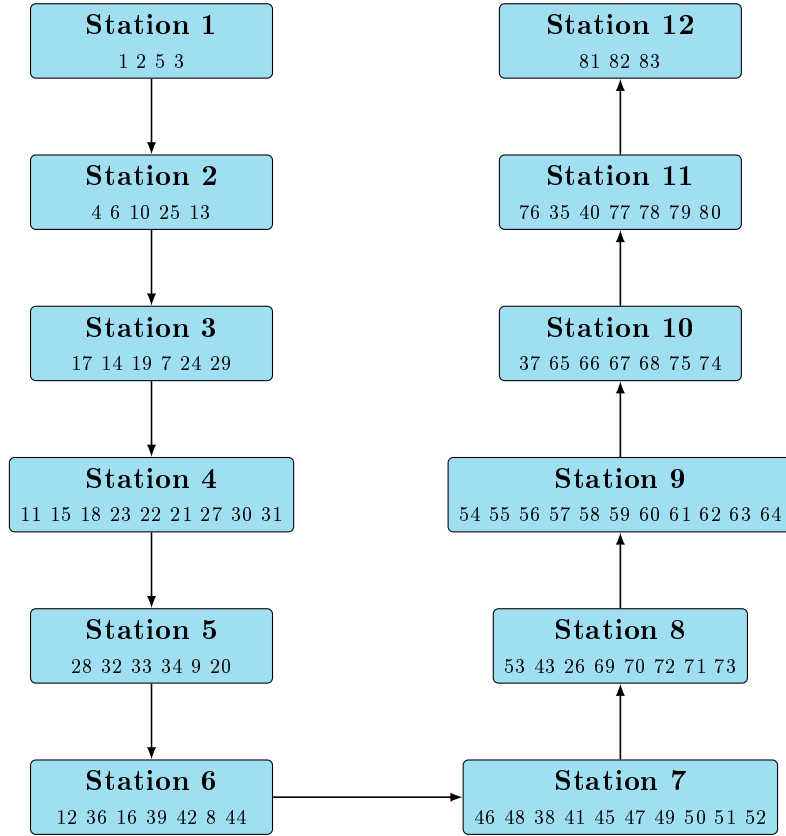


Figure 12: U-shaped 12-station assignment for ARC83 (Individual 5, Fitness = 0.000155).

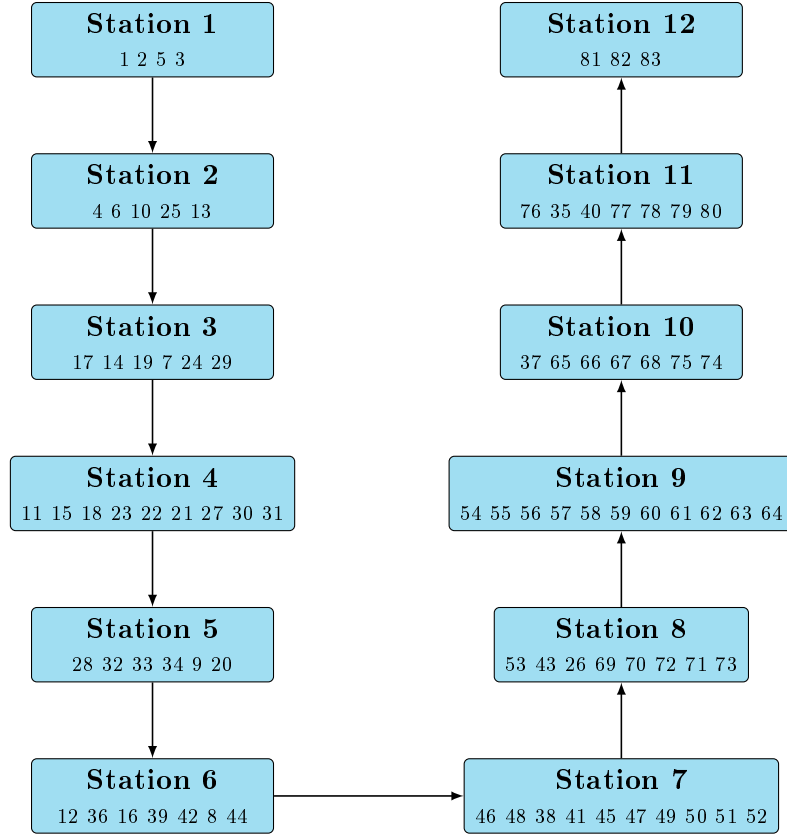


Figure 13: U-shaped 12-station assignment for ARC83 (Individual 6, Fitness = 0.000155).

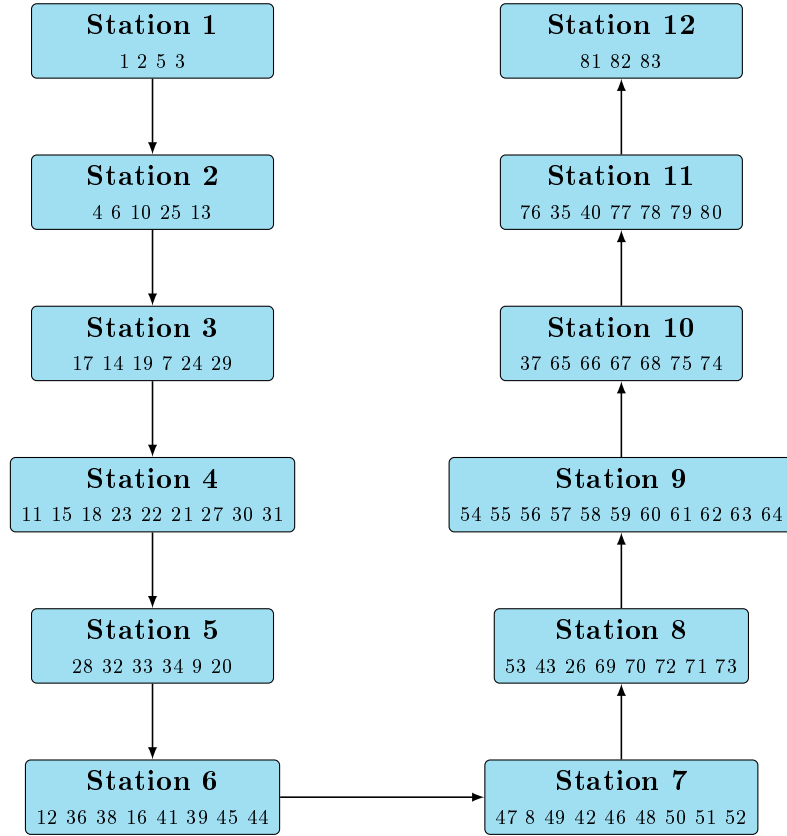


Figure 14: U-shaped 12-station assignment for ARC83 (Individual 7, Fitness = 0.000155).

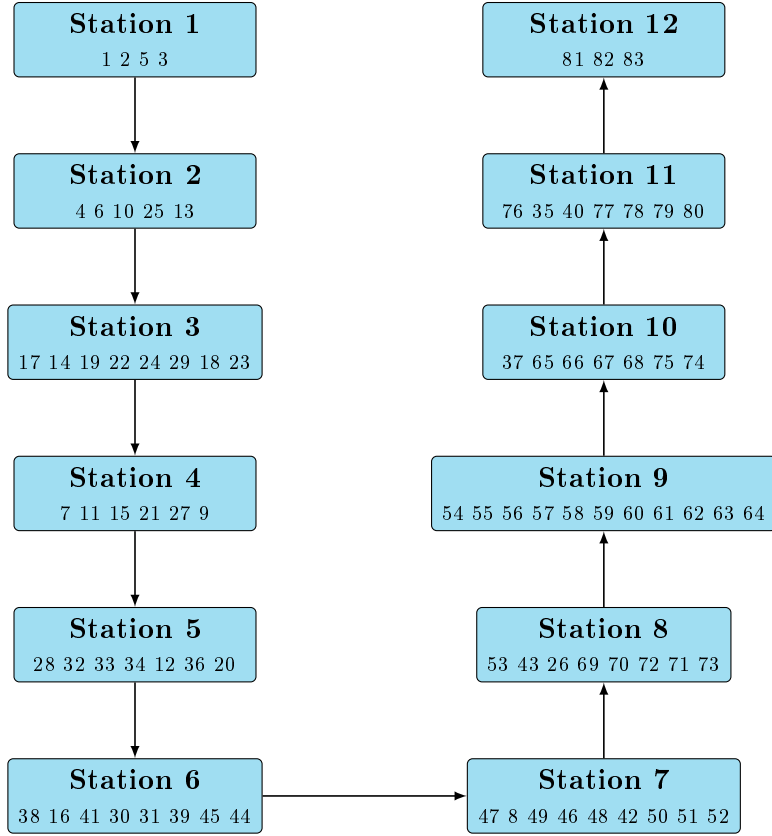


Figure 15: U-shaped 12-station assignment for ARC83 (Individual 8, Fitness = 0.000155).

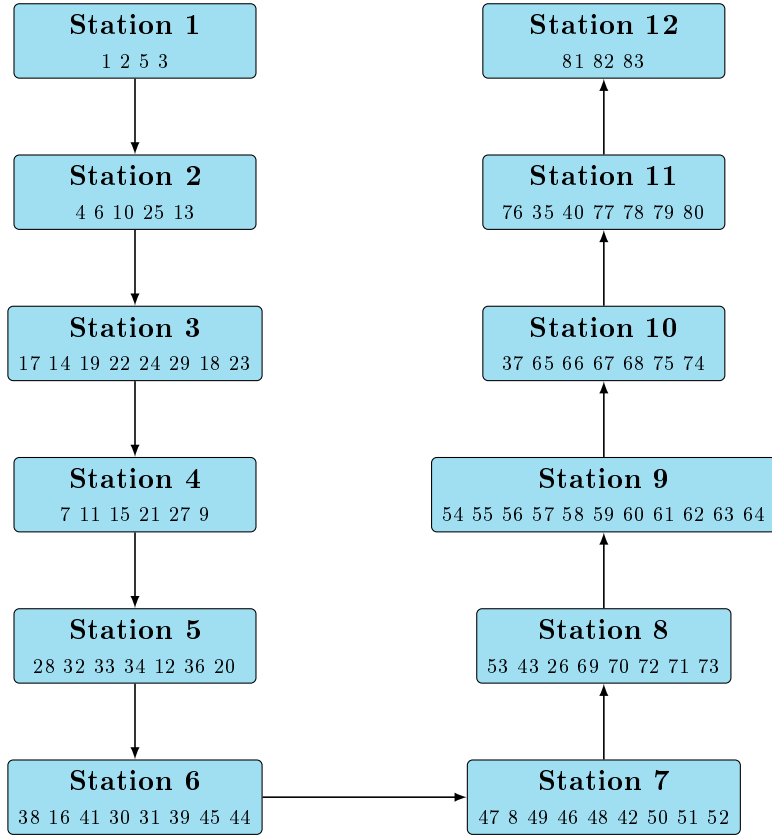


Figure 16: U-shaped 12-station assignment for ARC83 (Individual 9, Fitness = 0.000155).

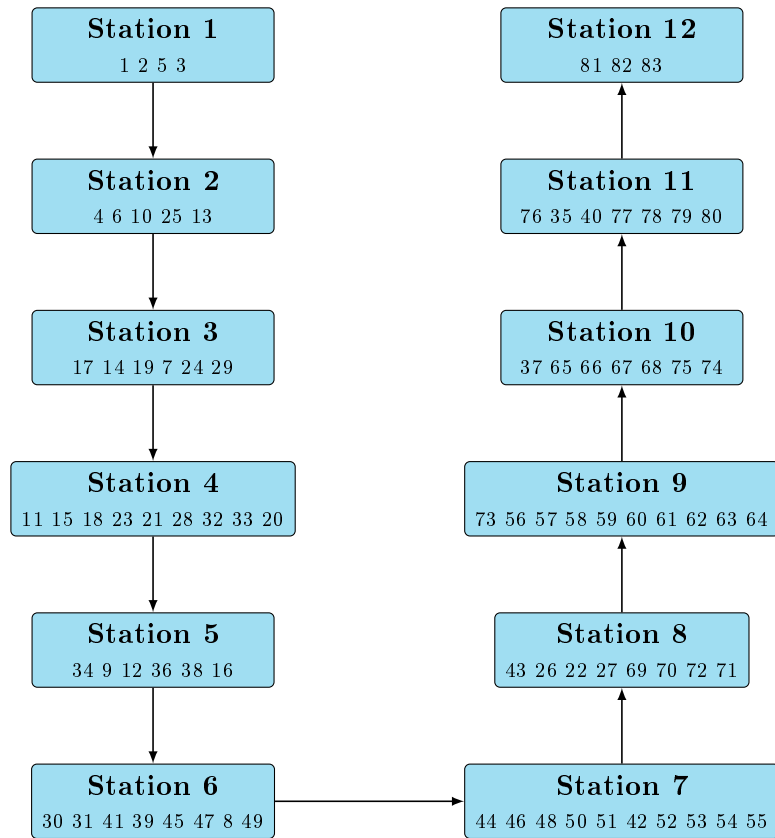


Figure 17: U-shaped 12-station assignment for ARC83 (Individual 10, Fitness = 0.000154).

D Line Graphs For 9 Instances

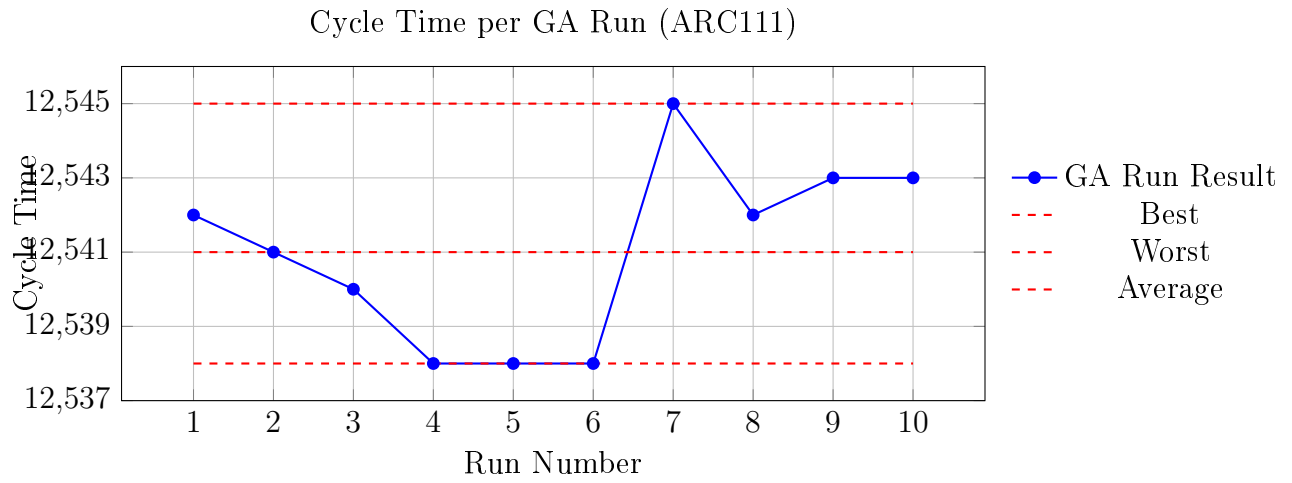


Figure 18: Instance ARC111 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

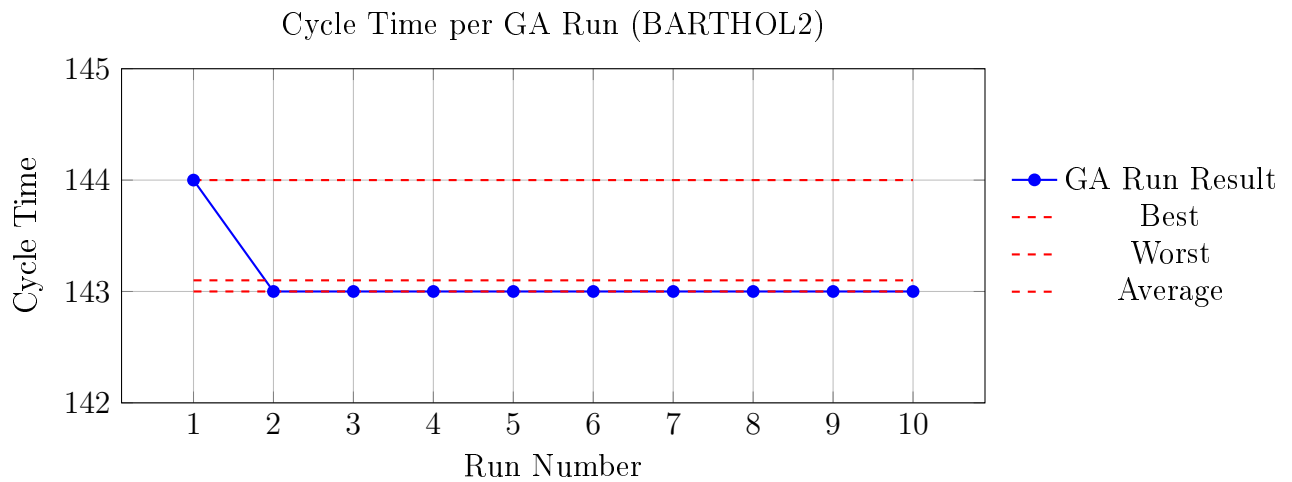


Figure 19: Instance BARTHOL2 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

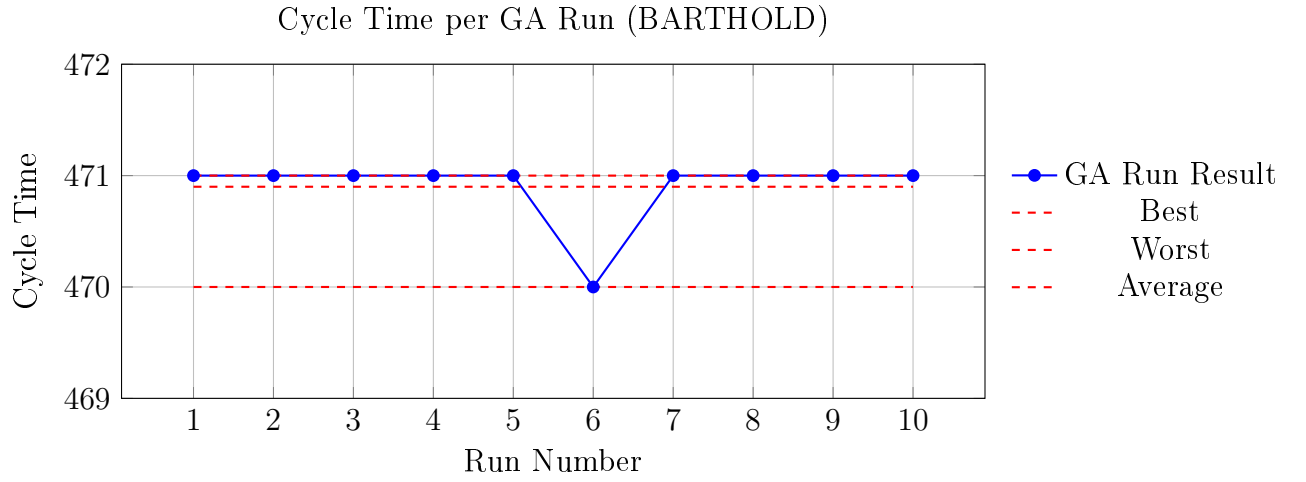


Figure 20: Instance BARTHOLD cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

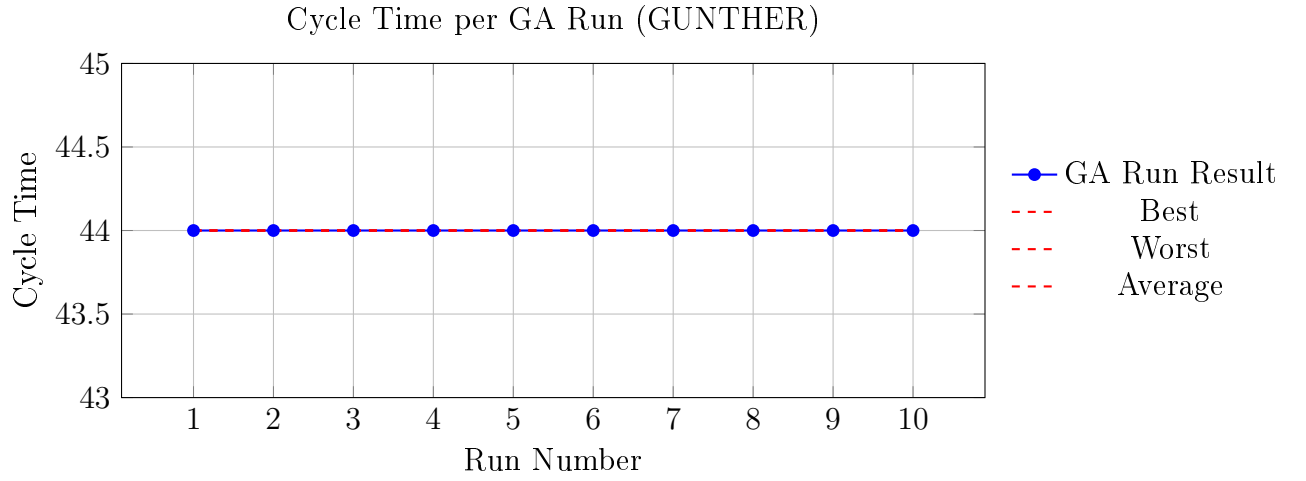


Figure 21: Instance GUNTHER cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

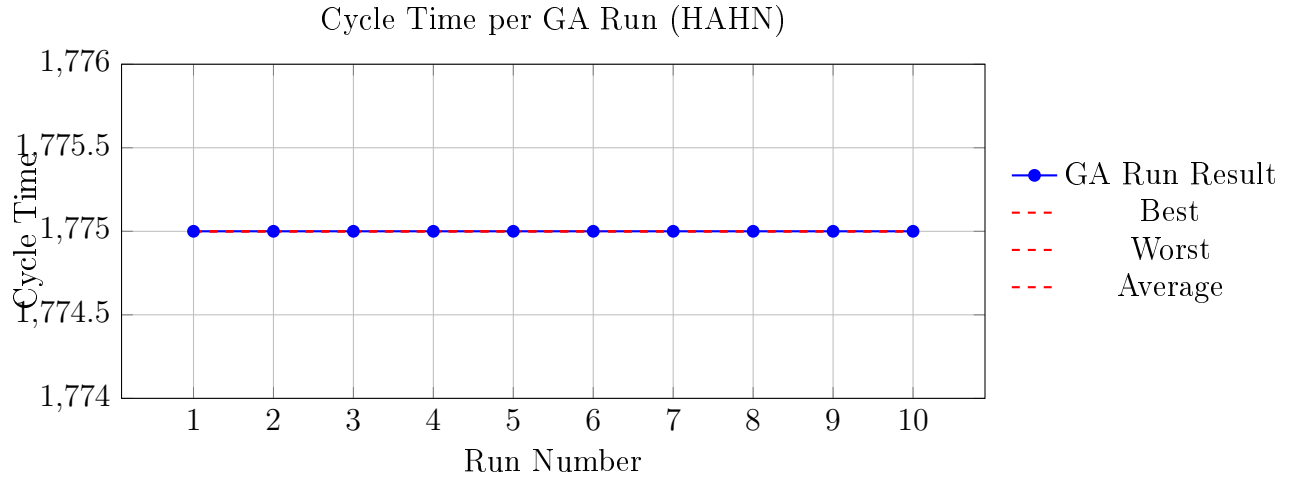


Figure 22: Instance HAHN cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

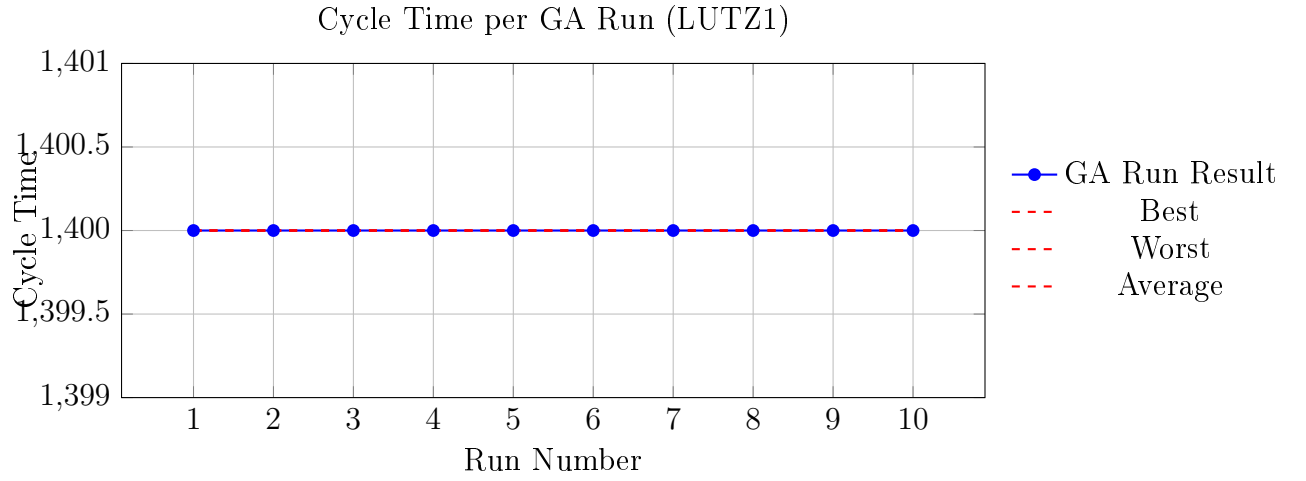


Figure 23: Instance LUTZ1 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

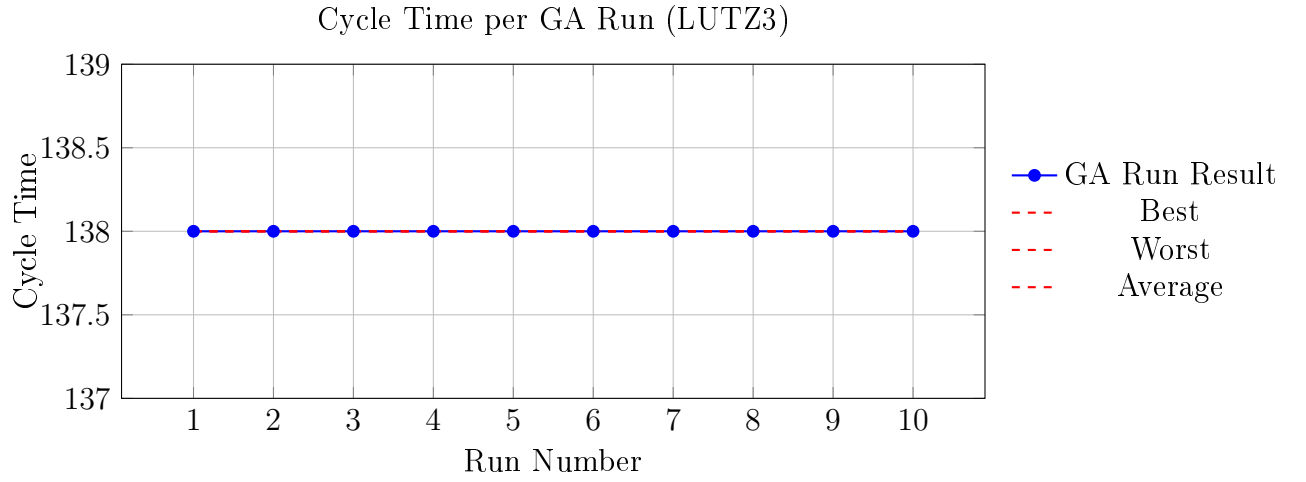


Figure 24: Instance LUTZ3 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

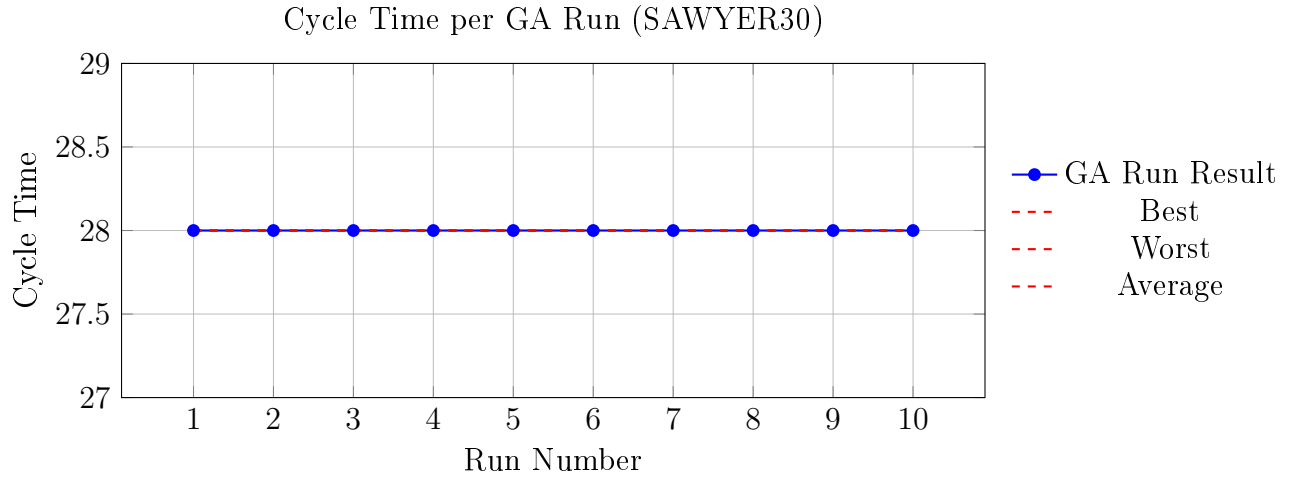


Figure 25: Instance SAWYER30 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

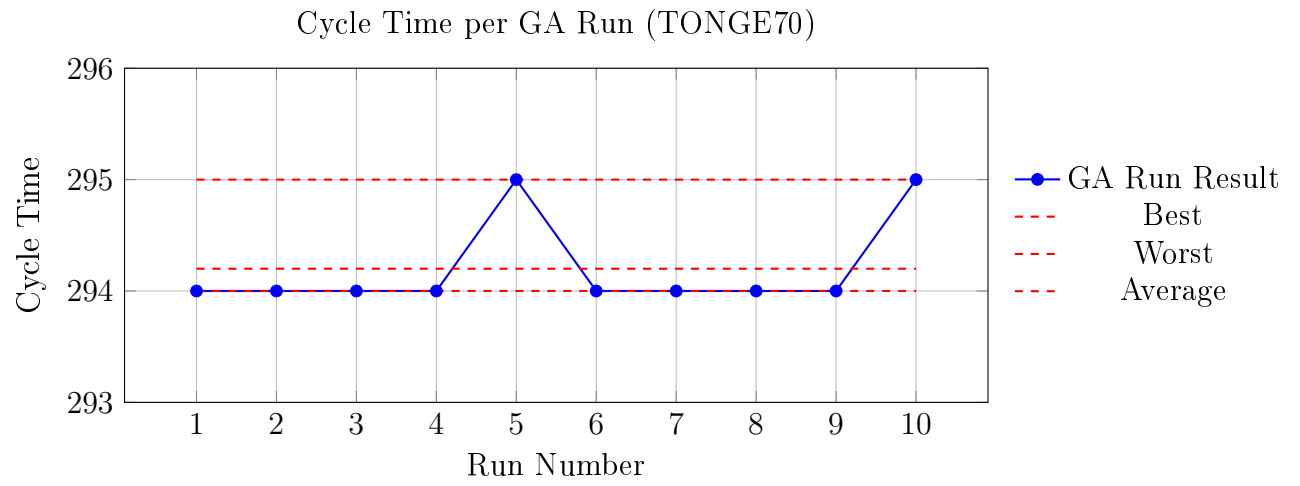


Figure 26: Instance TONGE70 cycle time obtained in each of the 10 independent GA runs, including best, worst, and average values.

E Genetic Algorithm Code for ARC83 (UALBP-II)

Listing 1: Python implementation of the genetic algorithm for the U-shaped assembly line balancing problem (UALBP-II) on the ARC83 instance.

```
1 import random
2 import math
3 import time
4 from typing import List, Tuple, Dict, Set
5 import csv
6 import os
7 from datetime import datetime
8
9 # =====
10 # KULLANICI AYARLARI
11 # =====
12
13 INSTANCE_PATH      = "ARC83.IN2"      # IN2 dosyanin yolu
14 M_STATIONS         = 12                # istasyon sayisi
15 POP_SIZE           = 100
16 GENERATIONS        = 250               # gorsellestirme - convergence
17 CROSSOVER_RATE     = 0.2
18 MUTATION_RATE      = 0.5
19 RUNS               = 10                # GA kac kez calisacak
20 BASE_SEED          = 42
21
22 OPTIMAL_CYCLE      = 138               # Excel'den bildigimiz optimum AMA
23     SALBP Icin
24
25 # =====
26 # POX DEBUG AYARLARI
27 # =====
28
29 DEBUG_POX          = False             # POX ornegi gormek istemezsen
30     False yap
31
32 POX_DEBUG_LIMIT     = 5                 # En fazla kac crossover ornegi
33     yazdirilsin
34
35 pox_debug_counter = 0
36
37 # =====
38 # 1) IN2 DOSYASINI OKUMA
39 # =====
```

```

35
36 def read_in2(path: str) -> Tuple[int, List[int], List[Tuple[int,
37     int]]]:
38     with open(path, "r") as f:
39         lines = [line.strip() for line in f if line.strip()]
40
41     n = int(lines[0])
42
43     times = []
44     for i in range(1, n + 1):
45         times.append(int(lines[i]))
46
47     precedences = []
48     for line in lines[n + 1:]:
49         if "," in line:
50             a, b = line.split(",")
51         else:
52             a, b = line.split()
53         i = int(a)
54         j = int(b)
55         if i == -1 and j == -1:
56             break
57         precedences.append((i, j))
58
59     return n, times, precedences
60
61 # =====
62 # 2) oNCuL / ARDIL KuMELERI
63 # =====
64
65 def build_pred_succ(
66     n: int,
67     precedences: List[Tuple[int, int]]
68 ) -> Tuple[Dict[int, Set[int]], Dict[int, Set[int]]]:
69     preds = {i: set() for i in range(1, n + 1)}
70     succs = {i: set() for i in range(1, n + 1)}
71
72     for i, j in precedences:
73         preds[j].add(i)
74         succs[i].add(j)

```

```

75
76     return preds, succs
77
78
79 # =====
80 # 3) TOPOLOJIK SIRALAMA + REPAIR
81 # =====
82
83 def random_topological_sort(
84     n: int,
85     preds: Dict[int, Set[int]],
86     rng: random.Random
87 ) -> List[int]:
88     remaining = set(range(1, n + 1))
89     result = []
90     assigned = set()
91
92     while remaining:
93         eligible = [i for i in remaining if preds[i].issubset(
94             assigned)]
95         if not eligible:
96             raise RuntimeError("Precedence graph contains a cycle?"
97                                )
98         chosen = rng.choice(eligible)
99         result.append(chosen)
100         assigned.add(chosen)
101         remaining.remove(chosen)
102
103     return result
104
105 def repair_to_topological(
106     perm: List[int],
107     preds: Dict[int, Set[int]]
108 ) -> List[int]:
109     rank = {task: idx for idx, task in enumerate(perm)}
110     remaining = set(perm)
111     result = []
112     placed = set()
113
114     while remaining:

```

```

114         eligible = [i for i in remaining if preds[i].issubset(
115             placed)]
116         if not eligible:
117             eligible = list(remaining)
118             chosen = min(eligible, key=lambda x: rank[x])
119             result.append(chosen)
120             placed.add(chosen)
121             remaining.remove(chosen)
122
123         return result
124
125     # =====
126     # 4) U-SHAPED DECODE + CYCLE TIME
127     # =====
128
129     def is_task_eligible(
130         task: int,
131         assigned: Set[int],
132         preds: Dict[int, Set[int]],
133         succs: Dict[int, Set[int]]
134     ) -> bool:
135         return preds[task].issubset(assigned) or succs[task].issubset(
136             assigned)
137
138     def decode_with_cycle_limit(
139         perm: List[int],
140         times: List[int],
141         preds: Dict[int, Set[int]],
142         succs: Dict[int, Set[int]],
143         m: int,
144         c: int
145     ) -> Tuple[bool, List[List[int]], List[int]]:
146         n = len(perm)
147         assigned: Set[int] = set()
148         stations: List[List[int]] = [[] for _ in range(m)]
149         loads: List[int] = [0 for _ in range(m)]
150
151         station_idx = 0
152

```

```

153 while station_idx < m and len(assigned) < n:
154     progress = True
155     while progress and len(assigned) < n:
156         progress = False
157         for task in perm:
158             if task in assigned:
159                 continue
160             if not is_task_eligible(task, assigned, preds,
161                                     succs):
162                 continue
163
164             duration = times[task - 1]
165             if loads[station_idx] + duration <= c:
166                 stations[station_idx].append(task)
167                 loads[station_idx] += duration
168                 assigned.add(task)
169                 progress = True
170
171             station_idx += 1
172
173     feasible = (len(assigned) == n)
174     return feasible, stations, loads
175
176 def evaluate_permutation_cycle_time(
177     perm: List[int],
178     times: List[int],
179     preds: Dict[int, Set[int]],
180     succs: Dict[int, Set[int]],
181     m: int
182 ) -> Tuple[float, List[List[int]], List[int]]:
183     total_time = sum(times)
184     max_time = max(times)
185
186     lb = max(max_time, math.ceil(total_time / m))
187     ub = total_time
188
189     best_c = None
190     best_stations = None
191     best_loads = None
192
193     while lb <= ub:

```

```

193         mid = (lb + ub) // 2
194         feasible, stations, loads = decode_with_cycle_limit(
195             perm, times, preds, succs, m, mid
196         )
197         if feasible:
198             best_c = mid
199             best_stations = stations
200             best_loads = loads
201             ub = mid - 1
202         else:
203             lb = mid + 1
204
205     if best_c is None:
206         return float("inf"), [[] for _ in range(m)], [0 for _ in
207             range(m)]
208
209     return float(best_c), best_stations, best_loads
210
211 # =====
212 # 5) GA OPERATORLERI
213 # =====
214
215 def tournament_selection(
216     population: List[List[int]],
217     fitness: List[float],
218     rng: random.Random,
219     k: int = 2
220 ) -> List[int]:
221     n = len(population)
222     best_idx = None
223     best_fit = -float("inf")
224     for _ in range(k):
225         i = rng.randrange(n)
226         if fitness[i] > best_fit:
227             best_fit = fitness[i]
228             best_idx = i
229     return population[best_idx][:]
230
231
232 def pox_crossover(

```

```

233     p1: List[int],
234     p2: List[int],
235     rng: random.Random
236 ) -> List[int]:
237     """
238     POX crossover + istege bagli debug ciktisi.
239     """
240     global pox_debug_counter
241
242     chosen = set()
243     for gene in p1:
244         if rng.random() < 0.5:
245             chosen.add(gene)
246     if not chosen:
247         chosen.add(rng.choice(p1))
248
249     child = []
250     for gene in p1:
251         if gene in chosen:
252             child.append(gene)
253     for gene in p2:
254         if gene not in chosen:
255             child.append(gene)
256
257     # ---- Debug output ----
258     if DEBUG_POX and pox_debug_counter < POX_DEBUG_LIMIT:
259         pox_debug_counter += 1
260         print("\n== POX CROSSOVER SAMPLE ==")
261         print("Parent1 (first 15):", p1[:15])
262         print("Parent2 (first 15):", p2[:15])
263         print("Chosen subset:", sorted(chosen))
264         print("Child (first 15):", child[:15])
265         print("=====\n")
266
267     return child
268
269
270 def swap_mutation(perm: List[int], rng: random.Random) -> List[int]
271 ]:
272     n = len(perm)
273     i, j = rng.sample(range(n), 2)

```

```

273     perm[i], perm[j] = perm[j], perm[i]
274     return perm
275
276
277 # =====
278 # 6) GA: TEK RUN + İSTATİSTİK
279 # =====
280
281 def genetic_algorithm_ualbp2(
282     n: int,
283     times: List[int],
284     preds: Dict[int, Set[int]],
285     succs: Dict[int, Set[int]],
286     m: int,
287     pop_size: int,
288     generations: int,
289     crossover_rate: float,
290     mutation_rate: float,
291     seed: int,
292     run_index: int
293 ):
294     rng = random.Random(seed)
295
296     population: List[List[int]] = []
297     for _ in range(pop_size):
298         perm = random_topological_sort(n, preds, rng)
299         population.append(perm)
300
301     best_perm = None
302     best_cycle = float("inf")
303     best_stations = None
304     best_loads = None
305     # ----- CONVERGENCE KAYDI (jenerasyon bazlı) -----
306     history_best_so_far = [] # her jenerasyon sonunda global best
307                             # (best_cycle)
308     history_best_gen = [] # her jenerasyonun kendi en iyisi
309     history_mean_gen = [] # jenerasyon ortalaması (populasyon)
310
311     for gen in range(generations):
312         fitness_vals = []

```

```

313     eval_cache = {}
314
315     for perm in population:
316         key = tuple(perm)
317         if key in eval_cache:
318             cycle_time = eval_cache[key]
319         else:
320             cycle_time, stations, loads =
321                 evaluate_permutation_cycle_time(
322                     perm, times, preds, succs, m
323                 )
324             eval_cache[key] = cycle_time
325             if cycle_time < best_cycle:
326                 best_cycle = cycle_time
327                 best_perm = perm[:]
328                 best_stations = stations
329                 best_loads = loads
330
331             if math.isinf(eval_cache[key]):
332                 fit = 0.0
333             else:
334                 fit = 1.0 / eval_cache[key]
335             fitness_vals.append(fit)
336
337     # ----- JENERASYON SONU CONVERGENCE METRIKLERI -----
338     # Bu jenerasyondaki cycle degerleri (eval_cache uzerinden)
339     gen_cycles = list(eval_cache.values())
340     gen_best = min(gen_cycles)
341     gen_mean = sum(gen_cycles) / len(gen_cycles)
342
343     history_best_gen.append(gen_best)
344     history_mean_gen.append(gen_mean)
345     history_best_so_far.append(best_cycle)
346
347     new_population: List[List[int]] = []
348     if best_perm is not None:
349         new_population.append(best_perm[:])
350
351     while len(new_population) < pop_size:
352         p1 = tournament_selection(population, fitness_vals, rng
353             , k=2)

```

```

352         p2 = tournament_selection(population, fitness_vals, rng
353                                     , k=2)
354
355         if rng.random() < crossover_rate:
356             child = pox_crossover(p1, p2, rng)
357         else:
358             child = p1[:]
359
360         if rng.random() < mutation_rate:
361             child = swap_mutation(child, rng)
362
363         child = repair_to_topological(child, preds)
364         new_population.append(child)
365
366         population = new_population
367
368         # ----- RUN SONU: final populasyon istatistikleri -----
369         final_stats = [] # (fitness, cycle, perm)
370         cycles = []
371         fits = []
372
373         for perm in population:
374             cycle_time, _, _ = evaluate_permutation_cycle_time(
375                 perm, times, preds, succs, m
376             )
377             fit = 0.0 if math.isinf(cycle_time) else 1.0 / cycle_time
378             cycles.append(cycle_time)
379             fits.append(fit)
380             final_stats.append((fit, cycle_time, perm))
381
382         best_cycle_final = min(cycles)
383         worst_cycle_final = max(cycles)
384         mean_cycle_final = sum(cycles) / len(cycles)
385
386         best_fit_final = max(fits)
387         worst_fit_final = min(fits)
388         mean_fit_final = sum(fits) / len(fits)
389
390         final_stats.sort(key=lambda x: x[0], reverse=True)
391         top10 = final_stats[:10]

```

```

392     # ek bilgi icin dondur
393     run_stats = {
394         "best_cycle": best_cycle,
395         "final_best_cycle": best_cycle_final,
396         "final_mean_cycle": mean_cycle_final,
397         "final_worst_cycle": worst_cycle_final,
398         "final_best_fit": best_fit_final,
399         "final_mean_fit": mean_fit_final,
400         "final_worst_fit": worst_fit_final,
401         "top10": top10,
402         "history_best_so_far": history_best_so_far,
403         "history_best_gen": history_best_gen,
404         "history_mean_gen": history_mean_gen,
405
406     }
407
408     return best_perm, best_cycle, best_stations, best_loads,
409           run_stats
410
411     # =====
412     # 7) cOKLU RUN + oZET
413     # =====
414
415     def main():
416         start_time = time.time()
417
418         # veri oku
419         n, times, precedences = read_in2(INSTANCE_PATH)
420         preds, succs = build_pred_succ(n, precedences)
421
422         total_time = sum(times)
423         max_time = max(times)
424         lb1 = math.ceil(total_time / M_STATIONS)
425         lb2 = max_time
426         lb = max(lb1, lb2)
427
428         print(f"Instance: {INSTANCE_PATH}")
429         print(f"Tasks (n)           : {n}")
430         print(f"Stations (m)           : {M_STATIONS}")
431         print(f"Total work content sum : {total_time}")

```

```

432     print(f"Max task time           : {max_time}")
433     print(f"Lower bound LB1=ceil(sum/m) = {lb1}")
434     print(f"Lower bound LB2=max(t_i)   = {lb2}")
435     print(f"Overall theoretical LB      = {lb}")
436     print()
437
438
439     all_convergence_rows = [] # tek dosyada tum run'leri
        birlestirmek icin
440     run_best_cycles = []
441     run_runtimes = []
442     best_overall_cycle = float("inf")
443     best_overall_solution = None
444     best_overall_top10 = None
445
446     for r in range(1, RUNS + 1):
447         seed = BASE_SEED + r
448         print(f"\n=====")
449         print(f"          RUN {r}/{RUNS}")
450         print(f"=====")
451         run_start = time.time()
452
453         best_perm, best_cycle, best_stations, best_loads, stats =
            genetic_algorithm_ualbp2(
454             n=n,
455             times=times,
456             preds=preds,
457             succs=succs,
458             m=M_STATIONS,
459             pop_size=POP_SIZE,
460             generations=GENERATIONS,
461             crossover_rate=CROSSOVER_RATE,
462             mutation_rate=MUTATION_RATE,
463             seed=seed,
464             run_index=r
465         )
466
467         hb = stats["history_best_so_far"]
468         hbg = stats["history_best_gen"]
469         hm = stats["history_mean_gen"]
470

```

```

471     for g in range(GENERATIONS):
472         all_convergence_rows.append([
473             r,                # Run index
474             g + 1,            # Generation
475             hb[g],            # Best-so-far
476             hbg[g],           # Best in generation
477             hm[g]             # Mean cycle
478         ])
479
480     run_end = time.time()                # bu run'in bitis
                                         zamanı
481     run_runtime = run_end - run_start    # sure (saniye)
482
483     run_best_cycles.append(best_cycle)
484     run_runtimes.append(run_runtime)
485
486     print(f"\nRun {r} summary:")
487     print(f"    Best cycle time over generations      : {
        best_cycle}")
488     print(f"    Final population best / mean / worst c: "
489           f"{stats['final_best_cycle']} / {stats['
        final_mean_cycle']:.2f} / {stats['
        final_worst_cycle']}]")
490     print(f"    Final population best / mean / worst fitness: "
491           f"{stats['final_best_fit']:.6f} / {stats['
        final_mean_fit']:.6f} / {stats['final_worst_fit
        ']:.6f}")
492
493     print("\n    Top 10 individuals in final population:")
494     for rank, (fit, cyc, perm) in enumerate(stats["top10"],
        start=1):
495         print(f"        {rank:2d}) fitness={fit:.6f}, cycle={cyc}")
496         # Ilgili istasyonlara dagilimi yazdir
497         feasible, stations, loads = decode_with_cycle_limit(
498             perm, times, preds, succs, M_STATIONS, int(cyc)
499         )
500         for s_idx, tasks in enumerate(stations, start=1):
501             print(f"            Station {s_idx}: {tasks}")
502
503     if best_cycle < best_overall_cycle:
504         best_overall_cycle = best_cycle

```

```

505         best_overall_solution = (best_perm, best_stations,
                                   best_loads)
506         best_overall_top10 = stats["top10"]
507
508     # 10 RUN oZET ISTATISTIKLER
509     mean_cycle_runs = sum(run_best_cycles) / len(run_best_cycles)
510     worst_cycle_runs = max(run_best_cycles)
511
512     best_gap_abs = best_overall_cycle - OPTIMAL_CYCLE
513     best_gap_pct = 100.0 * best_gap_abs / OPTIMAL_CYCLE
514
515     mean_gap_abs = mean_cycle_runs - OPTIMAL_CYCLE
516     mean_gap_pct = 100.0 * mean_gap_abs / OPTIMAL_CYCLE
517
518     total_runtime = time.time() - start_time
519
520     print("\n=====")
521     print("          10 RUN GLOBAL SUMMARY")
522     print("=====")
523     print(f"GA parameters:")
524     print(f"  Population size      : {POP_SIZE}")
525     print(f"  Generations           : {GENERATIONS}")
526     print(f"  Crossover rate        : {CROSSOVER_RATE}")
527     print(f"  Mutation rate         : {MUTATION_RATE}")
528     print(f"  Parent selection      : Tournament (k=2)")
529     print(f"  Crossover method      : POX-benzeri precedence preserving
530           ")
531     print(f"  Mutation method       : Swap mutation")
532     print(f"  Iteration number      : {GENERATIONS} generations per run
533           ")
534     print(f"  Total runs            : {RUNS}")
535     print()
536     print(f"Run best cycle times: {run_best_cycles}")
537     print(f"Best cycle over runs : {best_overall_cycle}")
538     print(f"Mean cycle over runs : {mean_cycle_runs:.2f}")
539     print(f"Worst cycle over runs : {worst_cycle_runs}")
540     print()
541     print(f"Known optimal C*      : {OPTIMAL_CYCLE}")

```

```

542 print(f"Best-of-10 gap          : {best_gap_abs} ({best_gap_pct
      :.3f} % above optimum)")
543 print(f"Mean-of-10 gap          : {mean_gap_abs:.2f} ({
      mean_gap_pct:.3f} % above optimum)")
544 print()
545
546 # En iyi cozumun istasyon yukleri ve gorevleri
547 best_perm, best_stations, best_loads = best_overall_solution
548 print("Best-of-10 solution station loads:")
549 for i, load in enumerate(best_loads, start=1):
550     print(f"  Station {i}: load = {load}")
551
552 print("\nBest-of-10 solution assignments:")
553 for i, tasks in enumerate(best_stations, start=1):
554     print(f"  Station {i}: {tasks}")
555
556 print("\nTop 10 individuals of best run (by fitness):")
557 for rank, (fit, cyc, perm) in enumerate(best_overall_top10,
      start=1):
558     print(f"  {rank:2d}) fitness={fit:.6f}, cycle={cyc}")
559     feasible, stations, loads = decode_with_cycle_limit(
560         perm, times, preds, succs, M_STATIONS, int(cyc)
561     )
562     for s_idx, tasks in enumerate(stations, start=1):
563         print(f"      Station {s_idx}: {tasks}")
564
565 print(f"\nTotal runtime for {RUNS} runs: {total_runtime:.2f}
      seconds")
566
567 # =====
568 # CSV OUTPUT: instance + timestamp ile isimlendirilmis dosyalar
569 # =====
570
571 instance_name = os.path.splitext(os.path.basename(INSTANCE_PATH
      ))[0]
572
573 # Tarih-saat etiketi (YYYY-MM-DD_HH-MM)
574 timestamp = datetime.now().strftime("%Y-%m-%d_%H-%M")
575
576 # Otomatik dosya adlari
577 summary_file = f"{instance_name}_summary_{timestamp}.csv"

```

```

578     runs_file          = f"{instance_name}_run_details_{timestamp}."
        csv"
579     best_details_file = f"{instance_name}_best_details_{timestamp}."
        csv"
580     top10_file         = f"{instance_name}-top-10-individuals-{
        timestamp}.csv"
581     convergence_file = f"{instance_name}_convergence_{timestamp}."
        csv"
582
583     print("\nGenerated output filenames:")
584     print("    Summary file          :", summary_file)
585     print("    Run details file       :", runs_file)
586     print("    Best details file      :", best_details_file)
587     print("    Top10 file             :", top10_file)
588     print()
589
590     # =====
591     # CSV OUTPUT 1: genel ozet
592     # =====
593     with open(summary_file, "w", newline="", encoding="utf-8") as f
        :
594         writer = csv.writer(f)
595
596         writer.writerow([
597             "Instance",
598             "Stations(m)",
599             "TotalWork(sum_ti)",
600             "Population",
601             "Generations",
602             "CrossoverRate",
603             "MutationRate",
604             "ParentSelection",
605             "CrossoverMethod",
606             "MutationMethod",
607             "Runs",
608             "BestCycle",
609             "MeanCycle",
610             "WorstCycle",
611             "OptimalC",
612             "BestGapAbs",
613             "BestGapPct",

```

```

614         "MeanGapAbs",
615         "MeanGapPct",
616         "TotalRuntime(sec)"
617     ])
618
619     writer.writerow([
620         instance_name,
621         M_STATIONS,
622         total_time,
623         POP_SIZE,
624         GENERATIONS,
625         CROSSEVER_RATE,
626         MUTATION_RATE,
627         "Tournament(k=2)",
628         "POX",
629         "Swap",
630         RUNS,
631         best_overall_cycle,
632         f"{mean_cycle_runs:.2f}",
633         worst_cycle_runs,
634         OPTIMAL_CYCLE,
635         best_gap_abs,
636         f"{best_gap_pct:.3f}",
637         f"{mean_gap_abs:.2f}",
638         f"{mean_gap_pct:.3f}",
639         f"{total_runtime:.2f}"
640     ])
641
642     print(f"CSV summary saved to: {summary_file}\n")
643
644     # =====
645     # CSV OUTPUT 2: run bazinda cycle & runtime
646     # =====
647     with open(runs_file, "w", newline="", encoding="utf-8") as f:
648         writer = csv.writer(f)
649
650         writer.writerow([
651             "Run",
652             "BestCycle",
653             "RunRuntime(sec)"
654         ])

```

```

655         for r in range(len(run_best_cycles)):
656             cyc = run_best_cycles[r]
657             rt = run_runtimes[r]
658             writer.writerow([r + 1, cyc, f"{rt:.2f}"])
659
660
661     print(f"Run details saved to: {runs_file}")
662
663     # =====
664     # CSV OUTPUT 3: en iyi cozumun istasyon detayi
665     # =====
666
667     total_line_eff = 100.0 * total_time / (M_STATIONS *
        best_overall_cycle)
668
669     with open(best_details_file, "w", newline="", encoding="utf-8")
        as f:
670         writer = csv.writer(f)
671
672         writer.writerow([
673             "Instance",
674             "Stations(m)",
675             "BestCycle",
676             "TotalWork(sum_ti)",
677             "GlobalLineEfficiency(%)",
678             "StationIndex",
679             "StationLoad",
680             "StationUtilization(Load/BestCycle)",
681             "TasksSequence"
682         ])
683
684     for s_idx, (load, tasks) in enumerate(zip(best_loads,
        best_stations), start=1):
685         utilization = load / best_overall_cycle
686         tasks_str = " ".join(str(t) for t in tasks)
687
688         writer.writerow([
689             instance_name,
690             M_STATIONS,
691             best_overall_cycle,
692             total_time,

```

```

693         f"{total_line_eff:.2f}",
694         s_idx,
695         load,
696         f"{utilization:.4f}",
697         tasks_str
698     ])
699
700     print(f"Best solution details saved to: {best_details_file}")
701
702     # =====
703     # CSV OUTPUT 4: Top 10 individuals of best run (her biri icin
704     # istasyonlar)
705     # =====
706     with open(top10_file, "w", newline="", encoding="utf-8") as f:
707         writer = csv.writer(f)
708
709         writer.writerow([
710             "Instance",
711             "Stations(m)",
712             "RankInBestRun",
713             "Fitness",
714             "Cycle",
715             "StationIndex",
716             "StationLoad",
717             "StationUtilization(Load/Cycle)",
718             "TasksSequence"
719         ])
720
721     for rank, (fit, cyc, perm) in enumerate(best_overall_top10,
722                                           start=1):
723         feasible, stations, loads = decode_with_cycle_limit(
724             perm, times, preds, succs, M_STATIONS, int(cyc)
725         )
726         for s_idx, (load, tasks) in enumerate(zip.loads,
727                                               stations), start=1):
728             tasks_str = " ".join(str(t) for t in tasks)
729             utilization = 0.0 if cyc == 0 else load / cyc
730
731             writer.writerow([
732                 instance_name,
733                 M_STATIONS,

```

```

731         rank ,
732         f"{fit:.6f}",
733         cyc ,
734         s_idx ,
735         load ,
736         f"{utilization:.4f}",
737         tasks_str
738     ])
739
740     print(f"Top 10 individuals (best run) saved to: {top10_file}")
741
742
743     # =====
744     # CSV OUTPUT 5: Convergence (tum run'lar, jenerasyon bazli)
745     # =====
746     with open(convergence_file, "w", newline="", encoding="utf-8")
747         as f:
748         writer = csv.writer(f)
749         writer.writerow(["Run", "Generation", "BestSoFar", "
750             BestInGeneration", "MeanCycle"])
751         writer.writerows(all_convergence_rows)
752
753     print(f"Convergence history saved to: {convergence_file}")
754
755     if __name__ == "__main__":
756         main()

```